

Verification Continuum™

Verdi® HW/SW Debug User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	6
Synopsys Statement on Inclusivity and Diversity	7

1. Introduction	8
Overview	8
Supported Platforms	9
Requirements	10
HW/SW Debug Flow	10
Compilation Phase	10
Compiler Optimizations	11
Simulation Phase	13
Converter Phase	14
Usage	14
Viewing Conversion Progress	19
Converting Cycle Counts into Time Values	19
Debug Phase	20

2. Invoking HW/SW Debug	21
Invoking HW/SW Debug in Hardware-Centric Flow	21
Debug Configurations Form	26
Main Tab	26
Architecture Tab	27
Advanced Tab	28
Invoking HW/SW Debug in Software-Centric Flow	29
Command Line Options	30

3. Using HW/SW Debug	32
Software Debug	33
Debugging Views	33
Simulation Time Display	33
Breakpoints	34
Debug Commands	36

Instruction Stepping Mode	37
Multiple Cores Debug	38
Active Core and Switching to Different Active Core	38
Thread Local Breakpoint Checking Mode	39
Synchronization With Hardware Debug	40
4. Performance Profiling	43
Generating Performance Profiler Reports	44
Generating Profiler Reports in Text Format	44
Viewing Profiler Reports in Eclipse	46
Gprof View	50
Generating Assembly Listing Annotated with Execution Times	51
Limitations	54
5. Coverage Support	55
Line Coverage Support	55
Generating Coverage Information	55
Options Supported by the hws_debug_coverage Tool	56
Merging Coverage Data	59
Requirements on ELF File	59
Viewing Line Coverage Information in Eclipse	60
Browsing Coverage Data at the File and Function Level in Eclipse	60
Viewing Covered/Uncovered Lines in Eclipse Source View	62
Generating a Text Report	63
Example of Text Report Containing Coverage Percentage per Function	63
Generating Annotated Source Files with Line Coverage Information	65
Example of Coverage Annotated Source File	66
Generating Annotated Disassembly File with Line Coverage Information	68
Example of Coverage Annotated Disassembly File	68
Coverage Exclusion Support	70
Specifying Exclusions	70
6. Adding a Custom Core	72
Prerequisites	72
Software Development Tool (Compiler)	72
Debugger (GDB)	73

Contents

HDL Simulator	73
Creating the HW/SW Debug Configuration File	73
Creating the Log-Pattern File	74
Testing and Debugging	76
Validating the Converter Pattern File	76
Invoking Verdi HW/SW Debug Tool	77
Integration Test	78
Appendix: Perl Script to Generate Register.XML File	78
7. Demo	80
Demo Example 1	80
Demo Example 2	83
8. Known Issues and Limitations	87

Preface

Verdi HW/SW Debug User Guide describes the Verdi Hardware-Centric flow (HW)/ Software-Centric flow (SW) Debug Solution.

This chapter describes the following section:

- [Customer Support](#)
- [Synopsys Statement on Inclusivity and Diversity](#)

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customerfacing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

In this chapter:

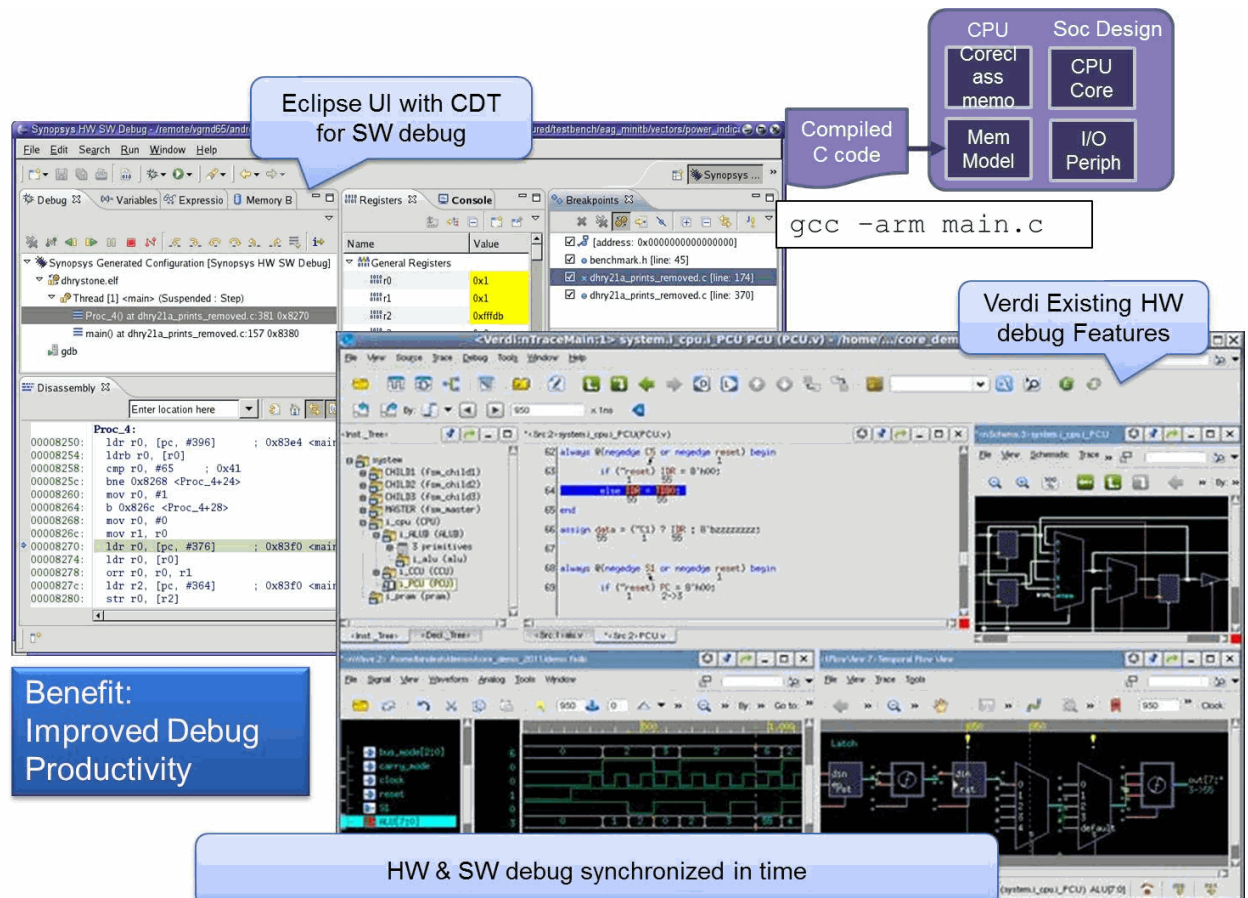
- [Overview](#)
- [HW/SW Debug Flow](#)

Overview

There are an increasing number of designs containing embedded processor cores. This implies that validation of these designs involves code running on the embedded cores, often being an integral part of the developed system. Likewise, development of test code is often part of the overall validation strategy. Hardware debug works at the hardware level where the hardware state is visualized as signals, whereas the embedded code is written in Assembly or C level. This leads to a gap in abstraction levels. Bridging the gap currently involves a solid understanding of the embedded core's workings. It is often done by developing ad hoc scripts to refine captured trace data to the assembly level. Going back to the C level can often only be achieved by tedious manual work.

The Verdi[®] Automated Debug Platform with the Verdi Hardware Software Debug (HW/SW Debug) Solution is a methodology of embedded software debugging where the embedded processor core under consideration is an HDL model and is a part of the simulated design. As illustrated in [Figure 1](#), the SoC design includes CPU cores, Memory models, I/O peripherals, and the compiled C image file is loaded into the memory model for core execution. The Verdi platform with HW/SW Debug integrates debug features between HDL and software to debug the design at the hardware level and the *Eclipse* program connects a Verdi plug-in to debug the embedded code at the Assembly or C level. The C/Assembly code, C variables, and stack are visible and the hardware and software debugging is synchronized in time. Designs with multiple cores can be debugged simultaneously.

Figure 1 HW/SW Debug Overview



You can view the behavior at the level of C or Assembly application source. In addition, it is much easier to understand the interaction between the software application and the hardware design.

Note:

Verdi HW/SW Debug supports all ARM Cortex cores out-of-the-box. Other cores, like, custom specific cores often can be easily supported. For details, contact the Synopsys Support Center (verdi_support@synopsys.com).

Supported Platforms

The Verdi HW/SW Debug Solution is only supported on *Linux RedHat* and *SUSE* platforms on 64 bit hosts. *Solaris* is not supported.

Requirements

To start running the Verdi HW/SW Debug Solution, **Verdi_HwSwDebug** license is required.

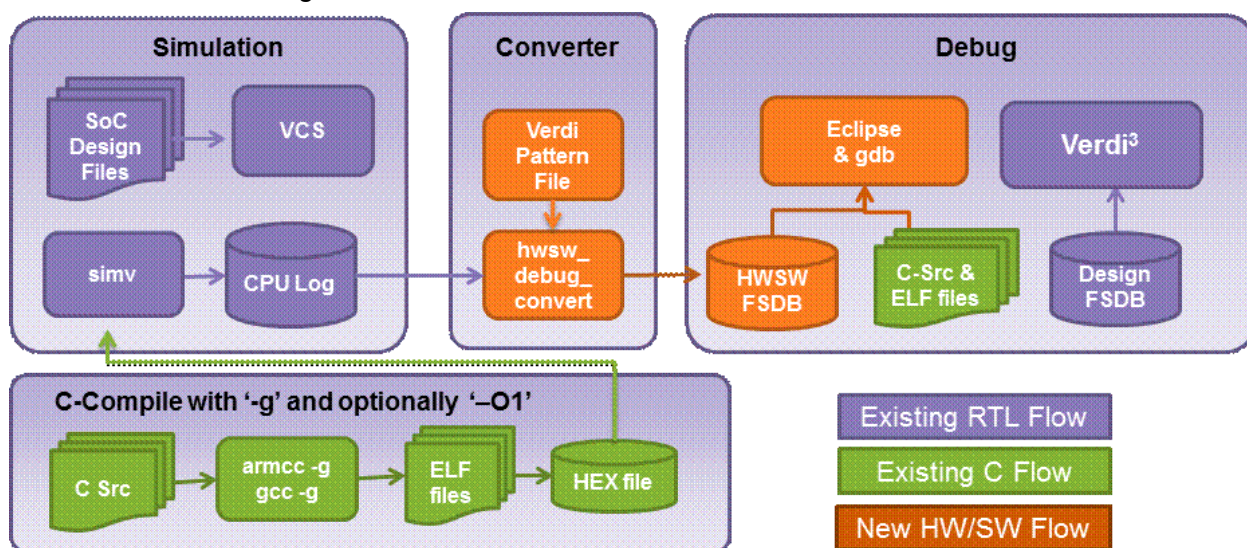
HW/SW Debug Flow

The following sections describe the flow of HW/SW debug solution:

- [Compilation Phase](#)
- [Simulation Phase](#)
- [Converter Phase](#)
- [Debug Phase](#)

[Figure 2](#) shows the HW/SW debug flow chart and specifies the required input files.

Figure 2 HW/SW Debug Flow



Compilation Phase

The CPU log file generated from the design (executing embedded C code) is used as an input by Verdi HW/SW Debug Solution. For details on generating this file, refer to your CPU vendor instructions.

The embedded application image file (ELF file) is created by *armcc*, *gcc*, or any other compiler. This ELF file is taken as an input by both the gdb engine and the Verdi HW/SW

Debug Solution to retrieve information about the initial memory state. To enable the compiler to generate debug information, such as, function name/memory address mapping, variable name/memory address mapping, filename and line number/memory address mapping, add the `-g` debug option to the compiler command, as follows:

```
gcc -g
armcc -g
```

Compiler Optimizations

Verdi HW/SW Debug Solution supports all C compiler optimization levels (`-O0`, `-O1`, `-O2`, and `-O3`). The default optimization level for `gcc` compiler is `-O0` and for `armcc` compiler is `-O2`.

Note:

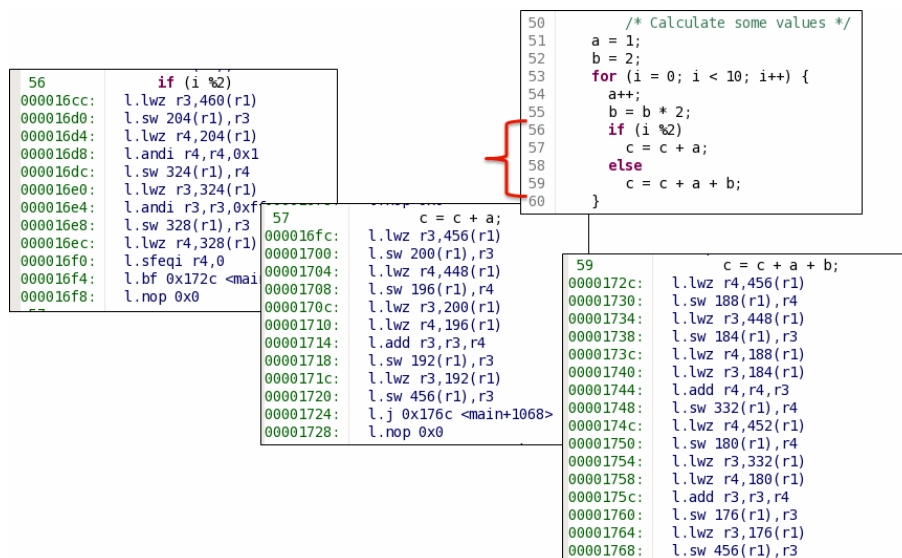
It is recommended to use `-Og` or `-O1` optimization option to improve the debug experience. However, all `-O` optimization levels (`-O0`, `-O1`, `-O2`, `-O3`) are supported.

The following sections show a sample code and the effect of using optimization options on this sample code.

Sample Code with -O0

In the sample code, each C source code line maps to many disassembly opcodes. With the `-O0` option, all the values are written to memory, and nothing is removed. This is also illustrated in [Figure 3](#).

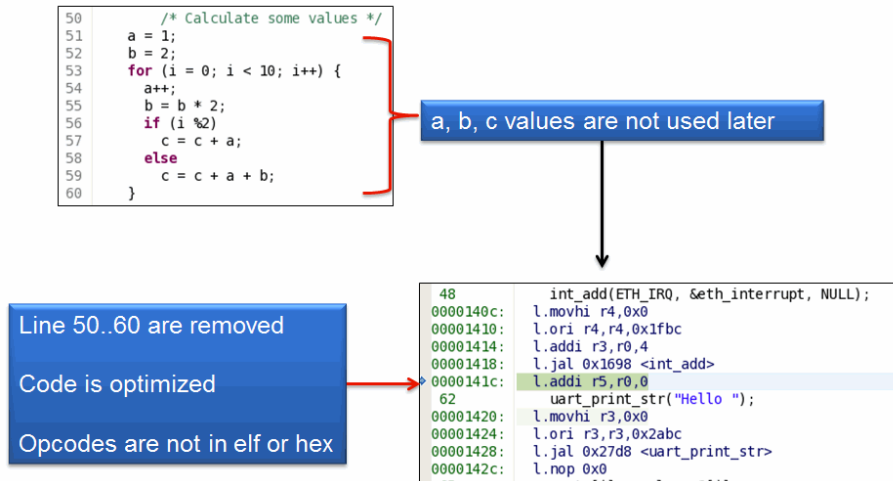
Figure 3 Sample Code with -O0



Sample Code with -O1

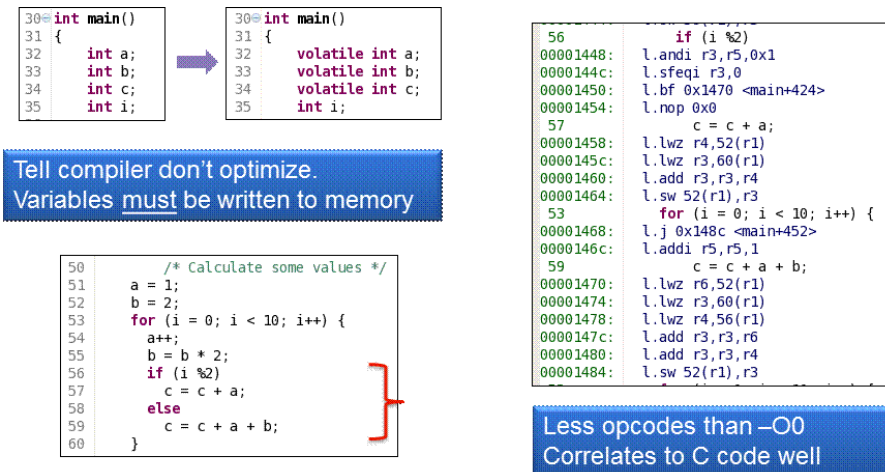
As illustrated in [Figure 4](#), the sample code is optimized, lines 50 through 60 are removed, and the variables are not written to memory.

Figure 4 Sample Code with -O1



When you instruct the compiler not to optimize the code and to write the variables to memory, the disassembly opcodes generated with the `-O0` option are lesser than those generated with the `-O1` option. Also, the disassembly code correlates to C code in a more effective way. This is also illustrated in [Figure 5](#).

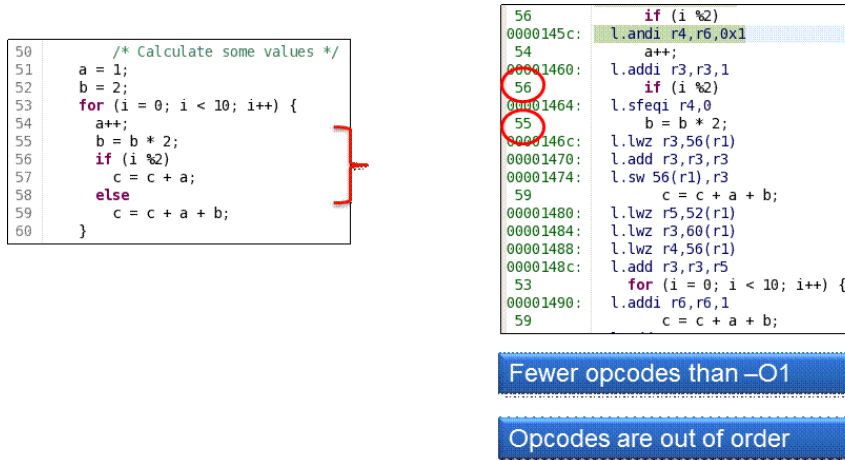
Figure 5 Sample Code with -O1 Without Optimization



Example: Sample Code with -O2

With the `-O2` option, the generated disassembly opcodes are lesser than those generated with the `-O1` option.

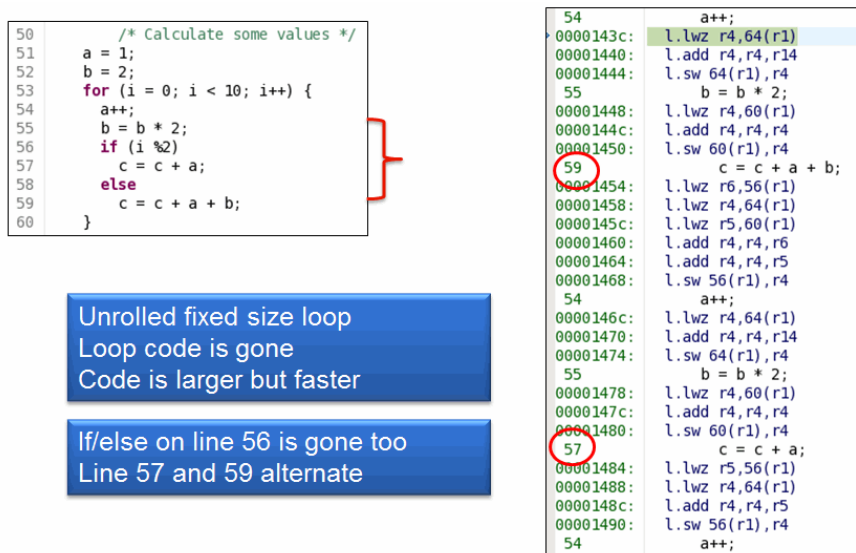
Figure 6 Sample Code with -O2



Example: Sample Code with -O3

With the -O3 option, the generated code is larger but faster. However, the variables are optimized, loops are unrolled, program counter changes, and variables with incorrect values are introduced. This is also illustrated in Figure 7.

Figure 7 Sample Code with -O3



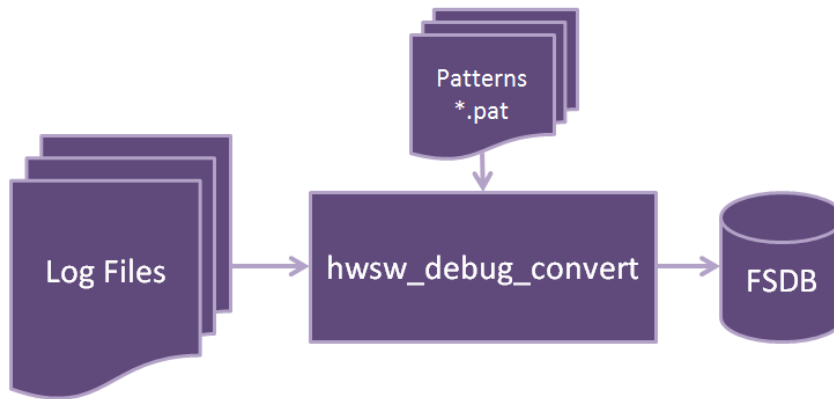
Simulation Phase

Enable log file dumping in the design before starting simulation. See CPU Vendor documentation for details.

Converter Phase

The `hws_debug_convert` tool directly converts one or multiple CPU log file(s) to an FSDB file containing data for the HW/SW Debug engine, as shown in the following figure:

Figure 8 HW/SW Debug Converter Flow



The configuration of the log file names, ELF file names, and CPU types is completed on the command line or in a design `.xml` file read by the `hws_debug_convert` tool.

The supported features of the `hws_debug_convert` tool are as follows:

- Read in one or multiple log files in configurable format.
- Write out data in an FSDB file.
- Support multi-core and multi-cluster designs.
- Support the log file format produced by all ARM[®] Cortex[®] cores. Log files produced by other CPUs are supported with minimal effort.
- Read in FSDB files for debugging purposes.
- Write out log file in generic format for debugging purposes.

Usage

To use the `hws_debug_convert` tool, specify the **`hws_debug_convert`** command in the command line, or create a `.xml` file that is read by the `hws_debug_convert` tool.

The following are examples of using the **`hws_debug_convert`** command to convert log files into FSDB files.

Note:

The parameter arguments (that is, arguments without a dash) of HW/SW Debug Converter must be specified before the options (that is, arguments with a dash).

- Converting a log file from an ARM Cortex-M3 processor into an FSDB file:

```
hws_debug_convert cpuType=Cortex-M3 exeFile=test.elf \
-i tarmac.log -o hws.fsdb
```

- Converting a log file from a four-core (ARM Cortex-A53), single-cluster system run in AArch64 mode into an FSDB file:

```
hws_debug_convert cpuType=Cortex-A53-64 exeFile=test.elf \
-num_cores=4 -i ca53_tarmac.0.0.%d.log -o hws.fsdb
```

- Converting a two-Cortex-A53, two-Cortex-A7, mixed-cluster system into an FSDB file:

```
hws_debug_convert cluster0.cpuType=Cortex-A53 \
cluster0.exeFile=test0.elf \
cluster0.traceFileName=ca53_tarmac.0.0.%d.log \
cluster1.cpuType=Cortex-A7 \
cluster1.exeFile=test1.elf \
cluster1.traceFileName=tarmac_cluster1_cpu%d.log \
-num_cores=2,2 -o hws.fsdb
```

Alternatively, this could be done in a design .xml file which is read in by the hws_debug_convert tool:

```
<cluster>
<name/>
<cluster>
<name>cluster0</name>
<core>
<name>cpu0</name>
<cpuType>Cortex-A53</cpuType>
<configName>AArch64</configName>
<exeFile>test0.elf</exeFile>
<traceFileName>ca53_tarmac.0.0.0.log</traceFileName>
<cpuId>0</cpuId>
<clusterId>0</clusterId>
</core>
<core>
<name>cpu1</name>
<cpuType>Cortex-A53</cpuType>
<configName>AArch64</configName>
<exeFile>test0.elf</exeFile>
<traceFileName>ca53_tarmac.0.0.1.log</traceFileName>
<cpuId>1</cpuId>
<clusterId>0</clusterId>
</core>
</cluster>
</cluster>
```

```
<name>cluster1</name>
<core>
  <name>cpu0</name>
  <cpuType>Cortex-A7</cpuType>
  <exeFile>test1.elf</exeFile>
  <traceFileName>tarmac_cluster1_cpu0.log</traceFileName>
  <cpuId>0</cpuId>
  <clusterId>1</clusterId>
</core>
<core>
  <name>cpu1</name>
  <cpuType>Cortex-A7</cpuType>
  <exeFile>test1.elf</exeFile>
  <traceFileName>tarmac_cluster1_cpu1.log</traceFileName>
  <cpuId>1</cpuId>
  <clusterId>1</clusterId>
</core>
</cluster>
</cluster>
```

Then, execute the following:

```
hwswh_debug_convert -load_design design.xml -o hwswh.fldb
```

Note that each cluster contains one or more cores and has its own memory space. In addition, each cluster launches a separate gdb (GNU Debugger).

Log Files

The log files are line-oriented text files and typically include information for instructions, register assignments and memory transactions. See the following for more details:

- Instruction information should include the program counter, and may also include timestamp, operation code and disassembly code.
- The register information should include the register specification (name or ID) and the value stored in the register.
- The memory information should include the address, data, data width and access type (read/write access).

The format of the log files depends on the model of the HDL processor. The following is an example of the log file:

```
10245 ns Ins 00000000 [e59ff018]
      Mem Read4 @(00000020)=0000003c
10975 ns Ins 0000003c [ea000011]
11445 ns Ins 00000088 [e3a00000]
      Reg R0=00000000
11445 ns Ins 0000008c [e3a01000]
      Reg R1=00000000
11445 ns Ins 00000090 [e3a02000]
      Reg R2=00000000
```

Command Line Switches/Parameters

This section lists the frequently used options and parameters of the **hwswh_debug_convert** command. The parameters may apply to a single core, multiple clusters, or all cores in the system. Refer to the [Usage](#) section for examples.

Following is the list of required options:

Required Switches	Description
-i filename	Specifies the input filename. The matching reader is selected based on the file extension. Text format is assumed for <i>.log</i> or <i>.txt</i> , and fsdb format is assumed for <i>.fsdb</i> .
-o filename	Specifies the output filename. The log file is written in a format derived from the file extension. Text format is assumed for <i>.log</i> or <i>.txt</i> , and fsdb format is assumed for <i>.fsdb</i> .
-load_designfile.xml	Loads SoC description from XML file. It is recommended to use this option for complex SoCs.

Note:

It is required to use either *-load_design* or *-i* and *-o* options.

Following is the list of optional options:

Optional Switches	Description
-num_cores N	Specifies the number of cores in a single cluster. This can also be a list of numbers separated by commas, which generates multiple clusters. It is recommended to use an XML file for multi-cluster designs.

Following is the list of required parameters:

Required Parameters	Description
exeFile=path	Specifies the name of the ELF executable for the coreLists multiple ELF files (if needed) in a ';' separated list.
cpuType=name	Specifies the core type. See <i>\$VERDI_HOME/share/hwswh_debug/patterns</i> .

Following is the list of optional parameters:

Optional Parameters	Description
bootExeFile= <i>path</i>	Specifies the name of the Boot ELF executable.
gdbPath= <i>path</i>	Specifies the path to GDB (usually set in architecture file).
patternFile= <i>file/path</i>	Specifies the name of the pattern file (usually set by cpuType).
traceFileName= <i>path</i>	Specifies the name of the input file (can be set by <code>-i <name></code>).
outFile= <i>path</i>	Specifies the name of the output file (can set by <code>-o <name></code>).
configName= <i>string</i>	Specifies the name of the architecture file. Required for custom cores only. Specify <i>ARMv7</i> or <i>ARMv7M</i> strings for ARM 32-bit cores and specify <i>AArch64</i> for ARM 64-bit cores.
Clock Parameters	Description
clockPeriod=	Specifies the clock period for cores with cyc or tic.
clockOffset=	Specifies the Offset for the cyc -> time conversion.
timeUnit=	Overrides the time unit in the CPU log file. Particularly useful if time-unit is wrong in log file.
fsdbScaleUnit=	Overrides the time unit in the generated FSDB file. Default is the time-unit in the CPU log, but sometimes you need it to match the design FSDB.
Debug Parameters	Description
printCorePrefix	Prints the core-name in text output.
printCycle	Prints the cycle-count in text output.

Pattern Files

The log file format is specified using trace patterns which are available in the Verdi release package at `$VERDI_HOME/share/hwsw_debug/patterns` location.

For example, for a Single Core R7,

```
% hwsw_debug_convert cpuType=Cortex-R7 exeFile=test.elf \
-i cpu.log -o hwsw.fsdb
```

The pattern files (*.pat* files) describe how the `hws_debug_convert` tool should read in the log files, in other words, the *.pat* file defines how a log file should be parsed.

Note:

Pattern files for supported ARM cores and other cores are supplied with Verdi HW/SW Debug. You should write a custom pattern file only for custom cores.

During parsing, each line in a log file is matched against all lines of the pattern file. If a line matches, the fields in the line is used to extract the data. The order in which the patterns are checked is not determined.

Each line of the pattern file is a regular expression, namely a Portable Operating System Interface (POSIX) extended regular expression.

For matching data, the lines also contain fields. The fields are identifiers which are enclosed by the '%' characters (such as `%PC%`), and determine the location of significant data for the `hws_debug_convert` tool.

The following is an example of the pattern file:

```
#PARAMETERS: configName=ARMv7
# Instructions:
%TIME% %TIMEUNIT% Ins %PC% \[%OPCODE%\] ( : %DISASM% ) ?
# Registers:
Reg %REGNAME%=%REGDATA%
# Memory:
Mem %RW%(ead|rite) %MEMSIZE% @\(%VADDR%\)=%MEMDATA%
```

Viewing Conversion Progress

The converter tool allows you to view the conversion progress using the progress bar. The progress bar is displayed when you run the `convert` command interactively from a terminal and if the conversion takes longer than a second. The progress bar is not displayed when the output of the `hws_debug_convert` tool is redirected to a file. You can use the `-progress yes|no|auto` option to force the progress bar on, or off.

Converting Cycle Counts into Time Values

If a CPU log file contains cycle count instead of time, you can use the following method to convert cycle counts into a time value:

- If the clock of the core is running at the same rate all the time, you can specify the frequency using the `clockPeriod` and `clockOffset` parameters.
- If the clock frequency of the core changes, or if the reset time is not constant, you cannot specify the `clockPeriod` and `clockOffset` parameters for conversion. In this case, you can use the following method to convert cycle counts into a time value:

During the run, the cycle counter signal of the core must be dumped into an FSDB file (for example, `design.fsdb` file). You must specify the `design.fsdb` file during the converter

run. You must also specify the cycle counter signal using the `cycleCounterSignal` parameter as shown in the following example.

Example:

```
hws_debug_convert -hw_fsdb design.fsdb  
cycleCounterSignal=top.dut.cluster0.cpu0.cycle_ counter_signal
```

Debug Phase

HW/SW Debug solution provides a synchronized view for embedded software debugging in Eclipse and HDL hardware design debugging in Verdi. *Eclipse* uses C source files, ELF file and FSDB file as input files whereas Verdi uses the design FSDB file as an input.

For example, use the following commands to debug your design:

```
% hws_debug -ssf hws.fsdb  
% hws_debug -ssf hws.fsdb -verdi_args "-lib -ssf design.fsdb"  
% hws_debug -ssf VirtualFile.vf
```

To create a Virtual File using Verdi, perform the following steps:

1. Open the *design.fsdb* file.
2. In the *nWave* window, select the **File -> Edit Virtual File** command.
3. Add the *hws.fsdb* file to *FSDB File list* using the **Add** button. Click **OK**.

The *VirtualFile.vf* file is created. This virtual file can then be used for future invocations of Verdi or *hws_debug*.

2

Invoking HW/SW Debug

The Verdi platform with HW/SW Debug provides the following flows to invoke HW/SW Debug:

- Hardware-Centric flow
- Software-Centric flow

In the Hardware-Centric flow, the *HW/SW Debug* window can be invoked from the Verdi platform to debug the design with both the hardware circuit and the embedded software.

In the Software-Centric flow, the *HW/SW Debug* window can be directly invoked to focus on the high-level embedded software of interest. If deeper investigation is needed for detailed hardware behavior, the Verdi platform can be invoked on demand.

The following sections describe how to invoke HW/SW Debug in hardware-centric and software-centric flows:

- [Invoking HW/SW Debug in Hardware-Centric Flow](#)
- [Invoking HW/SW Debug in Software-Centric Flow](#)

Invoking HW/SW Debug in Hardware-Centric Flow

The following steps illustrate how to invoke the *HW/SW Debug* window from the Verdi platform. Refer to the [Demo](#) section for an actual example.

1. Invoke the Verdi platform with the FSDB file:

```
> verdi -ssf hwsf.fsdb &
```

For complete debug of the HDL, the design files can also be loaded into the Verdi platform. Refer to the [Demo](#) section for an example.

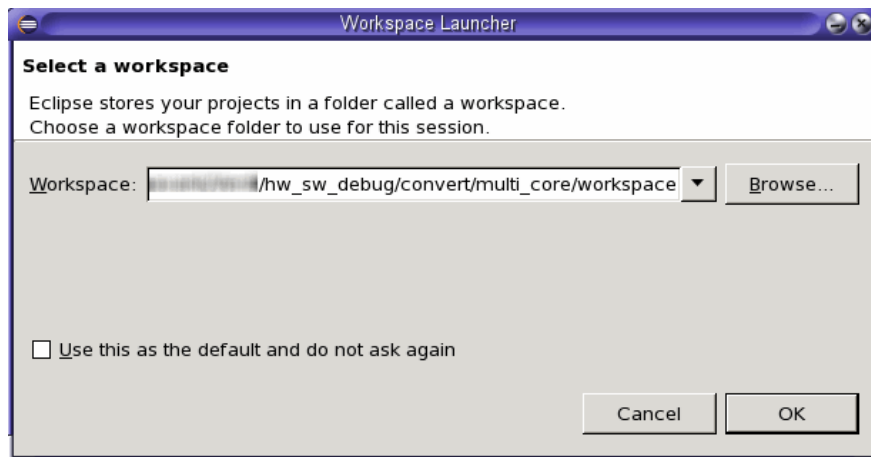
Select **Hardware Debug Mode** in the *Welcome* page. After loading the simulation results of the design including the ARM core into the Verdi platform, *Eclipse* with HW/SW Debug can be opened using the **Tools -> Invoke HW SW Debug** command. The *Workspace Launcher* form appears.

Note:

The *Eclipse* specified in the *HW/SW Debug* page of the *Preferences* form (invoked by using the **Tools -> Preferences** command) is executed. The default value is `$VERDI_HOME/bin/eclipse`.

2. Specify a workspace directory to store the *Eclipse* debug project and click the **OK** button.

Figure 9 *Workspace Launcher*



3. The *Debug Configurations* form appears. *Main*, *Architecture* and *Advanced* tabs are provided in this form for setting the debug session configuration.

Select the *Main* tab in the *Debug Configurations* form to configure HW/SW Debug session.

Select the *Architecture* tab to configure the viewing layout of the whole system under debug session. All the clusters and cores are listed in the *Architecture* tab. You can enable or disable particular clusters/cores for the debug session. The disabled clusters/cores are not considered for the corresponding HW/SW Debug session.

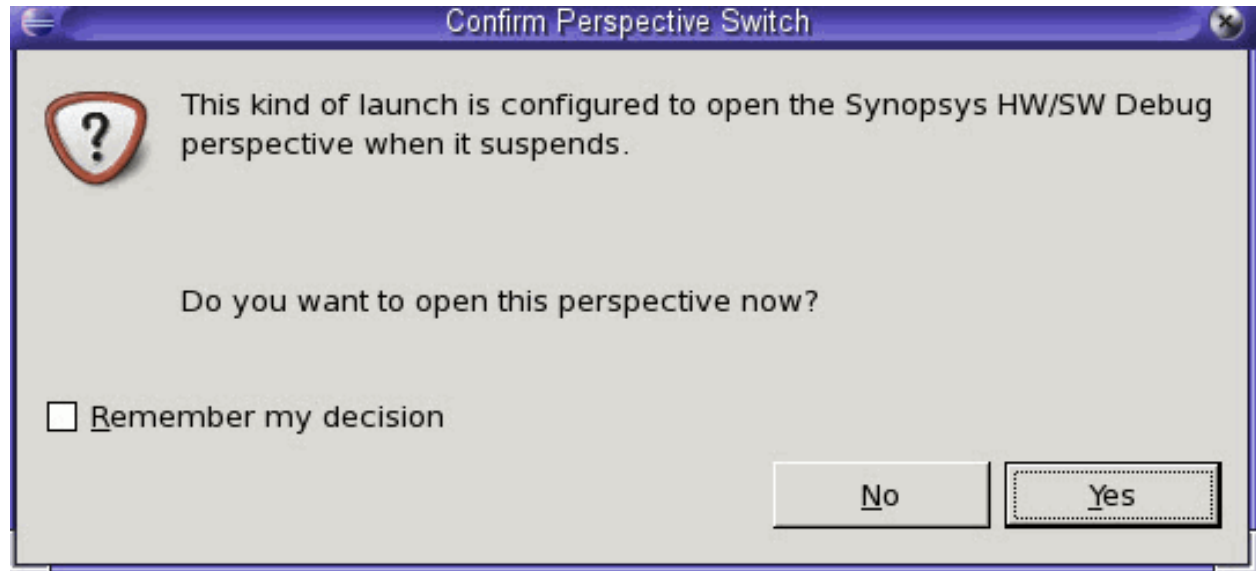
Select the *Advanced* tab to specify additional options. The options are sent to HW/SW Debug as the default value of advanced usage.

For details regarding the various tabs and options available in the *Debug Configurations* form, refer to the [Debug Configurations Form](#) section.

4. a. After specifying the debug configurations, click the **Debug** button in the *Debug Configuration* form. As shown in [Figure 10](#), when *Eclipse* starts for the first time, a confirmation dialog appears asking whether the *Synopsys HW SW Debug*

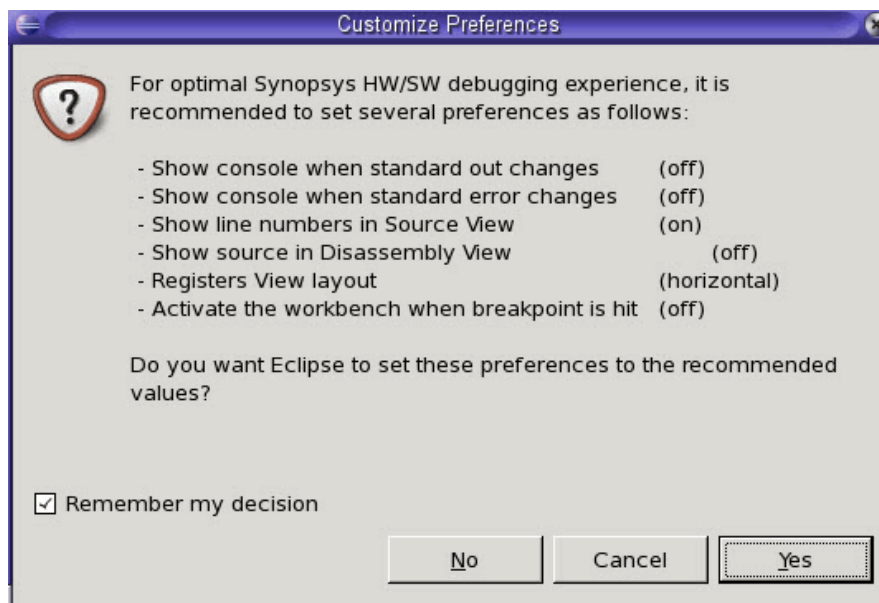
perspective should be made the default perspective in *Eclipse*. Click the **Yes** button to confirm the perspective switch.

Figure 10 Perspective Switch



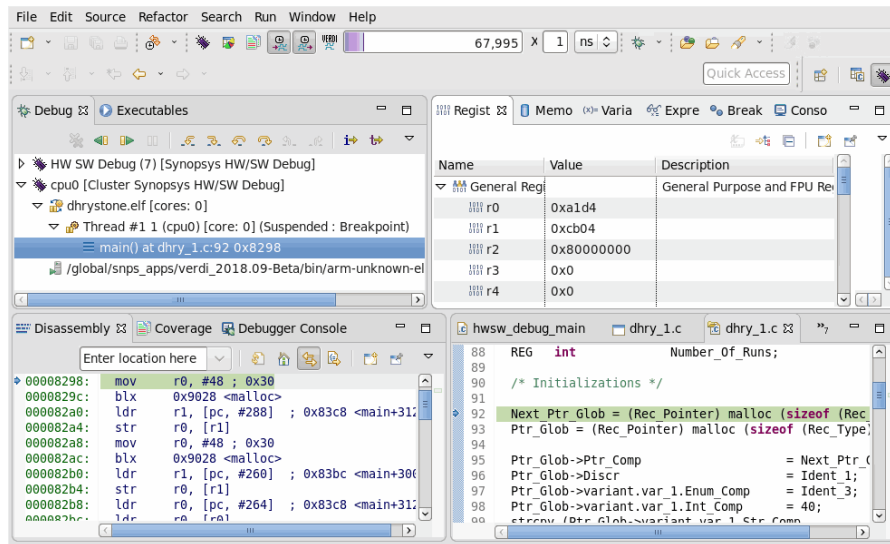
5. Then, click the **Yes** button in the *Customize Preferences* dialog to use the recommended values for the preferences.

Figure 11 Customize Preferences



6. In order to further augment the user experience and remove clutter, all the toolbars, menu items, and views that do not play any role in the *HW/SW Debug* window have been hidden in *Eclipse*. The default layout in *Eclipse* is from the perspective of HW/SW Debug.

Figure 12 Default Layout of HW/SW Debug Perspective in Eclipse



Starting with the 2018.09 Verdi release, the version of Eclipse used with the HW/SW Debug product is upgraded to Eclipse Neon. The following is a list of advantages of using Eclipse Neon:

- Eclipse Neon natively supports mapping of Windows paths to Linux paths stored within the ELF file during HW/SW Debug session.
- Eclipse Neon supports Virtualizer Platform tools.

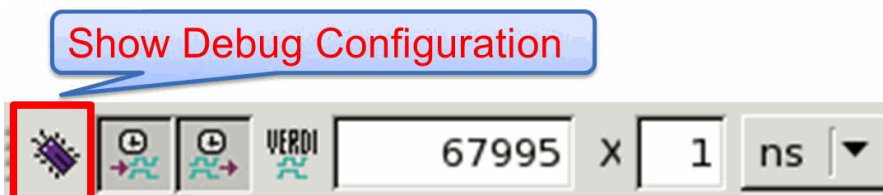
The following is the limitation of Eclipse Neon:

- Eclipse Neon does not support Embedded System Register Viewer plugin.

7. When *Eclipse* is launched from the Verdi platform, the installed HW/SW debug plugin reads the parameters from the *Eclipse* command line, creates a corresponding debug configuration and launches it. Therefore, you are automatically presented with a ready-to-go debugging environment. Use the **Run -> Debug Configurations** command or click the  icon in the *HW/SW Debug* toolbar (refer to [Figure 13](#)) to invoke the *Debug*

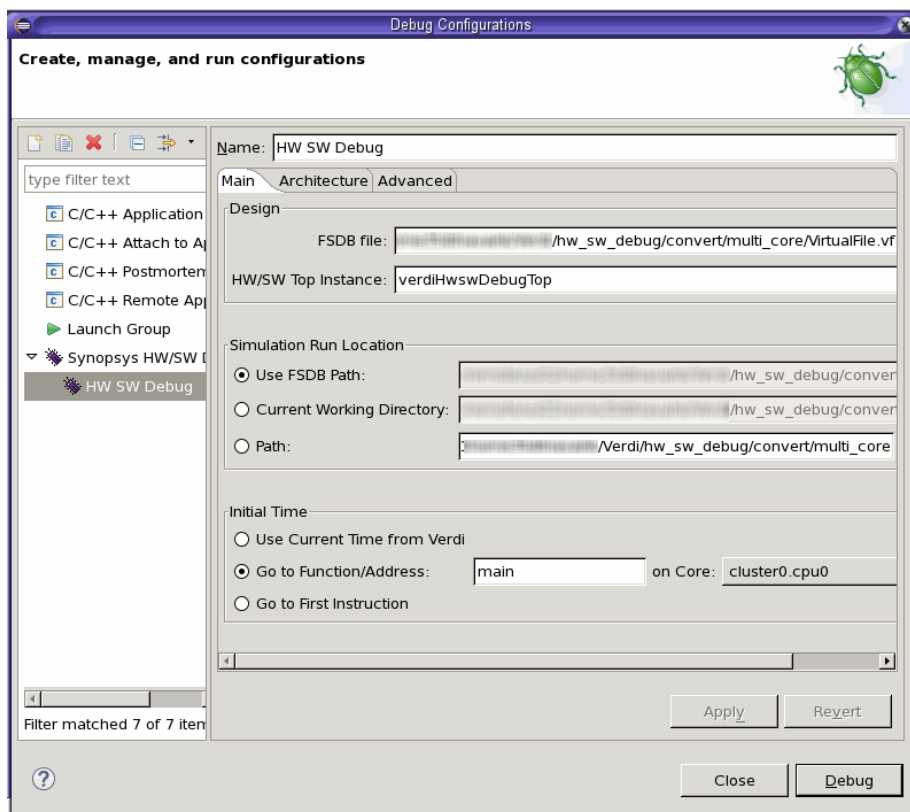
Configurations form to check and change the configuration. The changes are applied immediately without re-invoking *Eclipse* after clicking the **Debug** button.

Figure 13 Show Debug Configurations



As shown in [Figure 14](#), you can also directly add, copy, and delete individual debug configurations in a debug session.

Figure 14 Debug Configurations



Debug Configurations Form

The *Debug Configurations* form can be used to view and change all the parameters that affect the running of HW/SW Debug session. It contains **Main**, **Architecture**, and **Advanced** tabs and the following basic buttons on each of the tab:

- **Apply**

It triggers the standard apply mechanism in *Eclipse*. Additionally, it saves the architecture description to the *architecture files* specified in the *Architecture* tab in the *Debug Configurations* form.

- **Revert**

It triggers the standard revert mechanism in *Eclipse*. All changes in the *Debug Configurations* form are reverted.

- **Close**

It closes the *Debug Configurations* form and eventually prompts the user to save the changes. It does not affect the running debug session.

- **Debug/Continue**

Debug starts a new debug session.

Continue proceeds with the currently running session. If the user has changed something in the debug configuration, it prompts the user to save the changes. This does not affect the running debug session.

Main Tab

The **Main** tab is used to set the most important parameters of the HW/SW Debug configuration. The user can set the location of the FSDB file, specify the path to the HW/SW Top Instance in the design, and the run location of the simulation. The initial conditions of the HW/SW debug session can be controlled by choosing if the start should be at the current time from Verdi, go to a certain function, address in code on a certain core, or just run to the first instruction.

In case Verdi is not present in the system, then the **Use Current Time from Verdi** option is disabled and greyed-out.

The following sections are present in the *Main* tab:

- *Design*: Specifies the location of the FSDB file and the path of the *HW/SW Top Instance* in the design in the *FSDB* file and *HW/SW Top Instance* fields respectively.
- *Simulation Run Location*: You can specify the path where the simulation is executed by selecting one of the following toggle options: *Use FSDB Path*, *Current Working Directory*, or *Path*.

- **Initial Time:** Specify the initial conditions of the HW/SW Debug session by selecting one of the following toggle options:
 - *Use Current Time from Verdi:* The debug session starts at the current time of Verdi.
 - *Go to Function/Address:* The debug session goes to the specified function or address in source code on a certain core as the initial status.
 - *Go to First Instruction:* The debug session starts at the first instruction.

Figure 15 Main Tab: Debug Configurations Form

Name: HW SW Debug

Design

FSDb file: Browse

HW/SW Top Instance:

Simulation Run Location

☒ Use FSDb Path:

☐ Current Working Directory:

☐ Path: Browse...

Initial Time

☐ Use Current Time from Verdi

☒ Go to Function/Address: on Core:

☐ Go to First Instruction

Apply Revert

Close Debug

Architecture Tab

The layout of the whole system under debug can be viewed in the **Architecture** tab. The information displayed in the **Architecture** tab is a superposition of the information about the hierarchical organization coming from the FSDb file and the hierarchical information contained in the *architecture file*. The information in the *architecture file* always takes precedence, thus allowing the user to customize the hierarchical information.

Note:

The Cluster/Core parameters listed in the **Architecture** tab are read-only. To make changes in the parameters, re-run the conversion step using the `hwsd_debug_convert` tool.

To enable or disable particular Clusters/Cores for the debug session, check/uncheck the checkboxes in front of the Cluster/Core names. The disabled Clusters/Cores will not take part in the corresponding HW/SW Debug session.

To display the hierarchical organization of the clusters and their cores check the **Show only Cluster/Core hierarchy** option. This would help you to get a better overview of the architecture and to be able to quickly enable or disable whole clusters or single cores.

Figure 16 Architecture Tab: Debug Configurations Form

Name: HW SW Debug

Main Architecture Advanced

Architecture file: /home/verdi/.vscode/verdi/debug/design.xml Browse...

Parameter	Value
☒ core	cpu
cpuType	My ARM
configName	custom_recorder
gdbPath	arm-unknown-elf-gdb
exeFile	/Source/hwsw_debug/a15_dhrystone/power_indicative/dhrystone/elf/dhrystone.elf
cpuld	0
clusterId	0

☐ Show only Cluster/Core hierarchy

Apply Revert

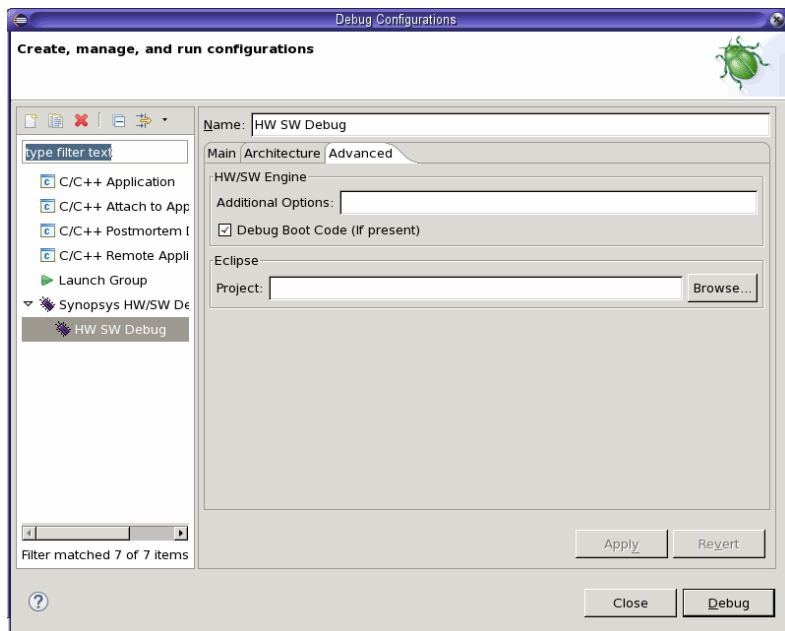
Close Debug

Advanced Tab

Additional options/parameters for advanced use can be specified in the **Additional Options** field in the **Advanced** tab. For example, the values for parameters that require more in-depth knowledge of the underlying system and/or change rarely from their default values can be specified here. The specified additional parameters will override the default startup parameters of *Eclipse* upon launch.

The project that is related to HW/SW Debug session can be added in the **Eclipse Project** field in the **Advanced** tab, if the project is already loaded into *Eclipse*.

Figure 17 Advanced Tab: Debug Configurations Form



Invoking HW/SW Debug in Software-Centric Flow

The following steps illustrate how to invoke HW/SW Debug in the Software-Centric flow. Refer to the [Demo](#) section for an actual example.

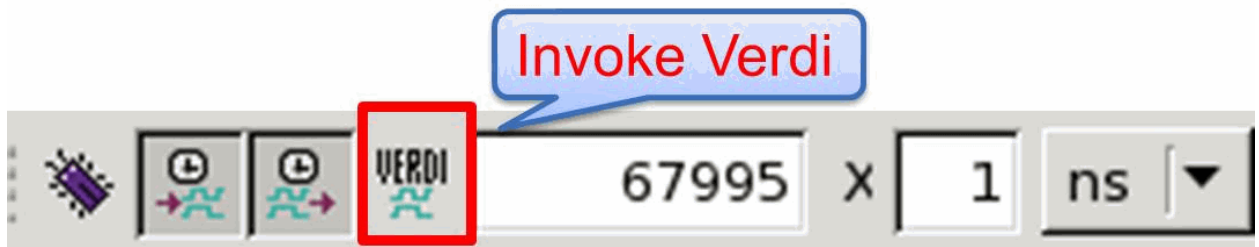
1. Invoke HW/SW Debug with the FSDB file using the `hwsd_debug` command:

```
hwsd_debug -ssf hwsd.fldb &
```

2. As mentioned in Step 2 of the [Invoking HW/SW Debug in Hardware-Centric Flow](#) section, the *Workspace Launcher* form appears. Follow the Steps 2 to 7 in the [Invoking HW/SW Debug in Hardware-Centric Flow](#) section to invoke the HW/SW Debug tool only.

Optionally, you can invoke the Verdi platform by clicking the  icon in the *HW/SW Debug toolbar*, as illustrated in [Figure 18](#).

Figure 18 Invoke Verdi from HW/SW Debug Toolbar



The `-ssf <FSDB_file>` option is automatically applied when launching the Verdi platform (that is, the FSDB file is opened in the *nWave* window as the active FSDB file).

You may also load a `design.fsdb` file into Verdi using the following command:

```
hws_debug -ssf hws.fsdb -verdi_args "-ssf design.fsdb"
```

You may also load a virtual file into Verdi using the following command:

```
hws_debug -ssf VirtualFile.vf
```

Simulation time in the Verdi platform is synchronized according to the value in the *Time* field in *Eclipse* and the setting of the *Synchronization* buttons (refer to the [Synchronization With Hardware Debug](#) section for time synchronization button details).

Command Line Options

The following table shows the most frequently used options for launching HW/SW Debug in the Software-Centric flow:

Option	Description
<code>-system_location <directory></code>	Specifies the path to which relative paths of simulation are relating to (such as, the path of the ELF file and source files). In most cases, this is the location where the simulation was run.
<code>-goto <point></code>	Specifies the initial point of <i>Eclipse</i> so that the active core stands at the address (such as, <code>-goto 0x0800</code>), label (such as, <code>-goto main</code>) or at the first instruction executed on any of the cores (<code>-goto @first_instruction</code>).
<code>active_core <core_name></code>	Specifies the core that is in focus initially and the cluster the <code>-goto</code> option applies to.

Option	Description
<code>-cores <core_name[,<core_name>]></code>	Limits the cores that are displayed while debugging. Lists multiple core names separated with ','. You can use this option to debug a subset of cores in a multicore system.
<code>-verdi_args <options></code>	Adds additional Verdi options that are passed to the Verdi platform when clicking the  icon to launch the Verdi platform.

Refer to the help of the `hwsd_debug` command for further option details:

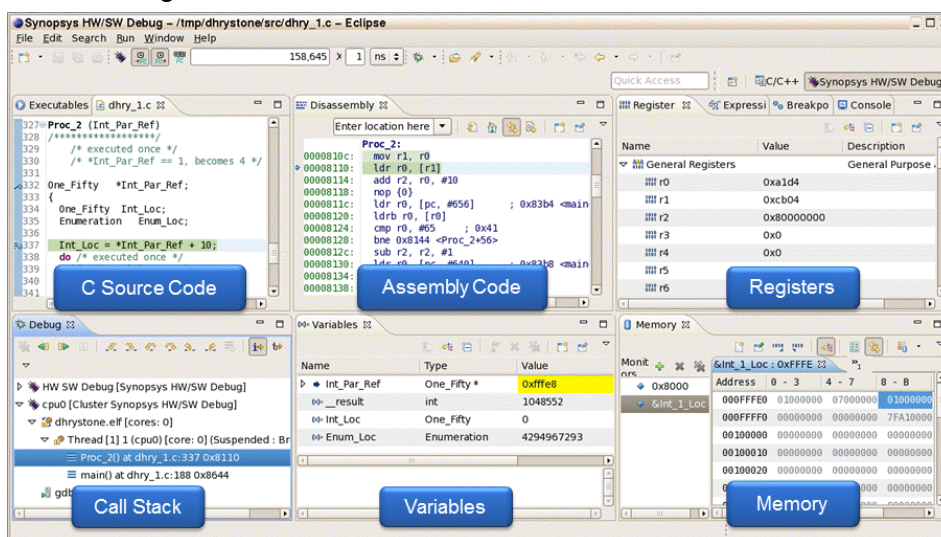
```
> hwsd_debug -help
```

3

Using HW/SW Debug

As a part of *Eclipse*, a customized *Synopsys HW SW Debug* perspective exists that is tailored to the needs of the HW/SW debug solution. Upon start of *Eclipse*, this perspective automatically becomes active allowing the best possible HW/SW debug experience. The HW/SW Debug tool provides a programmers view of C/Assembly code executed on HDL CPU code. As demonstrated in [Figure 19](#), contents of the core are displayed in separate corresponding views. The views can be added, reorganized, or removed and the repositioning of views is done by dragging the view by its title to the desired position.

Figure 19 Programmers View of Code on HDL CPU



The following sections describe the features of HW/SW Debug in *Eclipse* and how to synchronize between embedded software debugging and HDL hardware design in the Verdi platform.

- [Software Debug](#)
- [Multiple Cores Debug](#)
- [Synchronization With Hardware Debug](#)

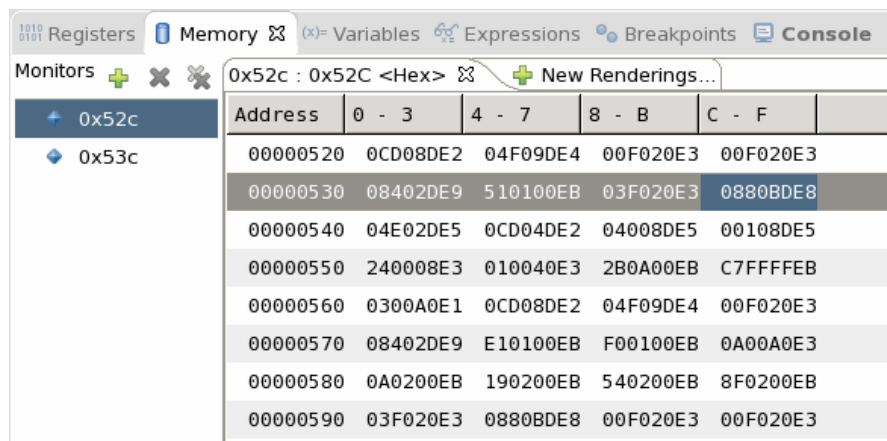
Software Debug

It is possible to debug embedded code in *Eclipse* using breakpoints, stepping, and running through the code. This means that debugging embedded software closely resembles the way software debugging usually works. In addition, HW/SW Debug provides the option to run and step backward in time. This enables easy answer to questions that frequently occur when debugging code, like "how did the program get here", "where does this value come from" or similar. So, going back to the call of the current function can be achieved with a single click.

Debugging Views

The location where the execution is currently stopped can be shown with a mark in the (C or Assembly) *Source Code* window during debugging (see [Figure 19](#)). The disassembly code is also shown with the current instruction marked in the *Disassembly* view. The general purpose registers of the core and their values are shown in the *Registers* view. The *Variables* view shows the values of C code variables. As demonstrated in [Figure 20](#), memory contents as seen by the core at that point in time are displayed in the *Memory View*.

Figure 20 Memory View



Address	0 - 3	4 - 7	8 - B	C - F	
00000520	0CD08DE2	04F09DE4	00F020E3	00F020E3	
00000530	08402DE9	510100EB	03F020E3	0880BDE8	
00000540	04E02DE5	0CD04DE2	04008DE5	00108DE5	
00000550	240008E3	010040E3	2B0A00EB	C7FFFFFFEB	
00000560	0300A0E1	0CD08DE2	04F09DE4	00F020E3	
00000570	08402DE9	E10100EB	F00100EB	0A00A0E3	
00000580	0A0200EB	190200EB	540200EB	8F0200EB	
00000590	03F020E3	0880BDE8	00F020E3	00F020E3	

Simulation Time Display

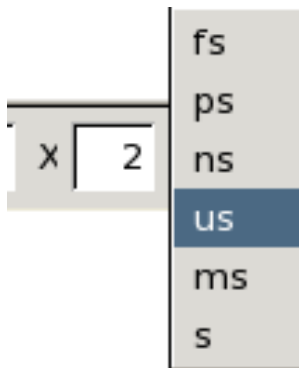
As demonstrated in [Figure 21](#), the current simulation time is displayed in the tool bar. There are two fields showing time value and time unit. Current simulation time is the product of time value and time unit.

Figure 21 Simulation Time Display in Eclipse



You can change the time unit (refer to [Figure 22](#)) or the value in the *Time Unit* field, and the value in the *Time Value* field gets automatically re-calculated. However, the current simulation time is not changed by changing the time unit.

Figure 22 Change Time Unit



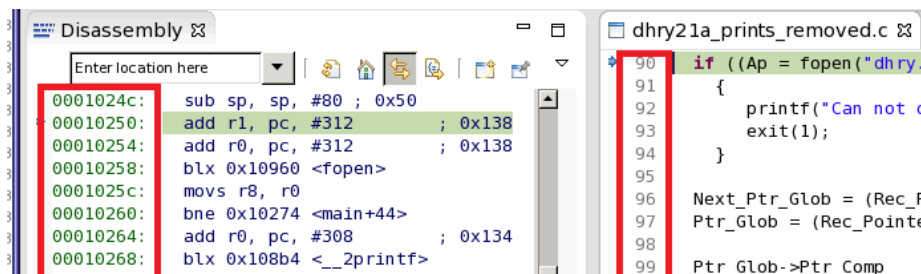
The value in the *Time Value* field can be changed to update the value in all the views in *Eclipse* according to the changed value. If time synchronization from Software (*Eclipse*) to Hardware (*Verdi*) is enabled, the new time is also propagated to the Verdi platform (refer to the [Synchronization With Hardware Debug](#) section for details).

Breakpoints

Breakpoints can be added on a given line of C or Assembly *Source Code* view and on a given address from within the *Disassembly* view. In [Figure 23](#), the area marked with red is where the user can perform right-click and create a breakpoint from the *context* menu

within the *Disassembly* and the *Source Code* views. The  or  icon is displayed on the line of the instruction or source code to indicate the breakpoint location.

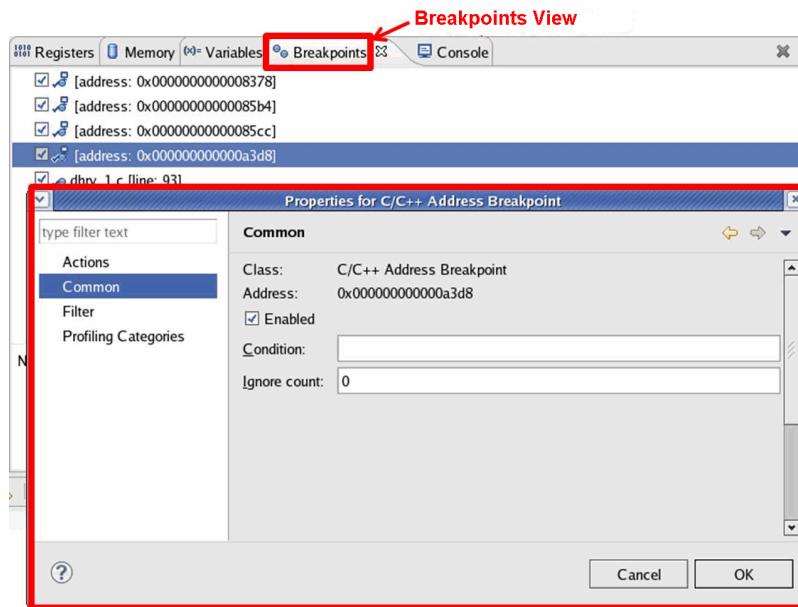
Figure 23 Areas Where Context Menu is Available to Create Breakpoint



All existing breakpoints in the debugging session are displayed in the *Breakpoint* view and the ☒ icon indicates that breakpoint is enabled (refer to [Figure 24](#)).

Click the ☒ or ☐ box to enable and disable breakpoints. You can right-click the breakpoint and choose commands to remove the breakpoint, select all breakpoints, or enable/disable all of them. In addition, choose the *Breakpoint Properties* command to add a condition or an ignore count to the breakpoint.

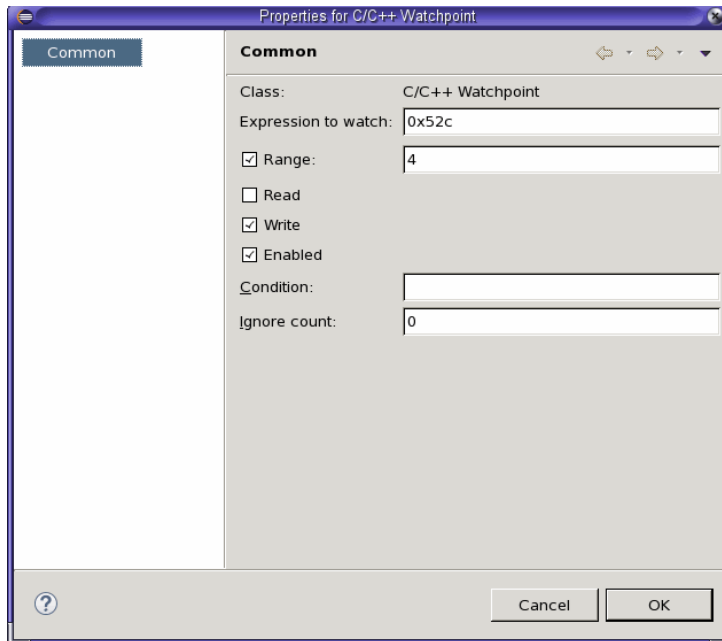
Figure 24 *Breakpoint Properties Dialog*



To create a memory breakpoint, right-click on the context menu of a memory cell in the *Memory View* and click the **Add Watchpoint (C/C++)...** option.

The *Properties for C/C++ Watchpoint* dialog opens up, as illustrated in [Figure 25](#). Click the ☒ icon next to **Read**, **Write**, or both the options to configure the memory breakpoint/watchpoint to be sensitive to read, write, or both the actions. For example, if **Read** is enabled, the execution stops when the watchpoint expression is read.

Figure 25 Properties for C/C++ Watchpoint Dialog



Debug Commands

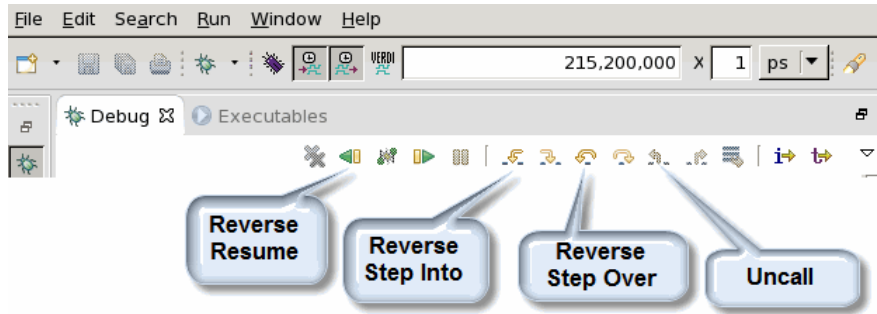
The standard debug commands can be issued by clicking one of the icons on the debug toolbar or using the corresponding menu items available under the **Run** menu. **Reverse Resume**, **Reverse Step Into**, **Reverse Step Over**, and **Uncall** commands are additionally provided in the debug toolbar to run or step backward in time.

- **Reverse Resume:** Run backward in time until a breakpoint condition is met.
- **Reverse Step Into:** Run backward in time until the next instruction or source line.
- **Reverse Step Over:** Run backward in time until the previous source line is reached. Skip over any subroutine calls encountered.
- **Uncall:** Run backward in time until the point where the current subroutine is called.

The *debug toolbar* can be found in the following two places:

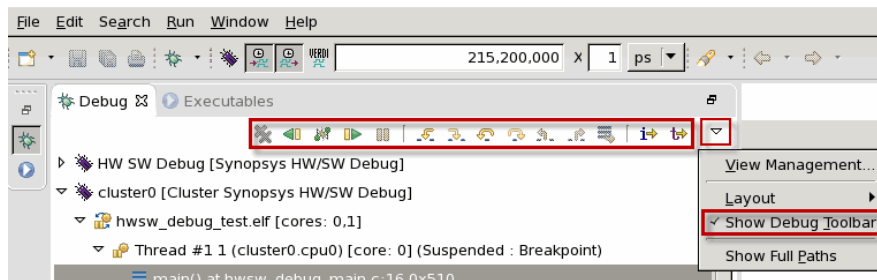
- By default, it is visible as part of the *main toolbar* (refer to [Figure 26](#)).

Figure 26 Debug Toolbar in the Main Window



- Alternatively, the *debug toolbar* can be activated as part of the *Debug* view by clicking the drop-down *context* menu in the *Debug* view and choosing the **Show Debug Toolbar** command. This is demonstrated in [Figure 27](#).

Figure 27 Debug Toolbar in Debug View



Every debug command is directly issued only to the currently active core. Refer to the [Multiple Cores Debug](#) section for details about how to identify the active core and how to switch to a different active core.

Instruction Stepping Mode

Usually the stepping commands are executed based on source lines in C/C++ or Assembly files. This means that stepping advances the simulation until the next or previous source line is reached. During debug, sometimes no source code exists, so debugging has to be done based on the disassembly code. In this case, normal stepping skips over the code for which no source code exists.

HW/SW Debug provides an *instruction stepping mode* to support debugging by stepping into instructions. In the *debug* view, click the  icon on the *debug toolbar* to enable

instruction stepping mode. After *instruction stepping mode* is enabled, the stepping commands are executed based on single assembly instructions.

Figure 28 Instruction Stepping Mode Icon

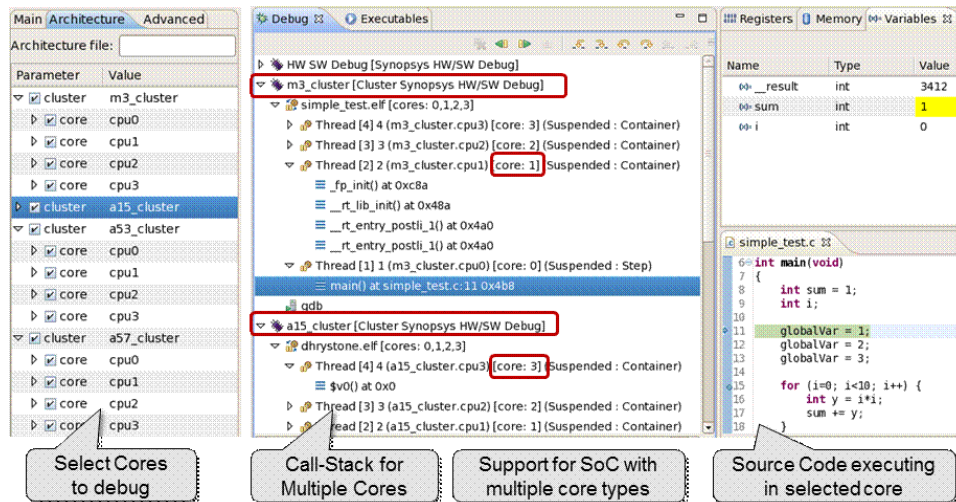


Multiple Cores Debug

HW/SW Debug solution provides support for debugging complex designs having multiple CPUs or multiple clusters wherein each cluster further comprises of multiple cores. HW/SW Debug solution provides programmer's views into multiple clusters and cores simultaneously and the debug view shows call stack for multiple clusters and cores. In the *Debug View*, the clusters are assigned a separate debugger process and the related cores are each assigned a separate debugging thread of the debugger process.

Figure 29 illustrates debug view for a multi-cluster multi-core design that has two clusters and each cluster contains four cores.

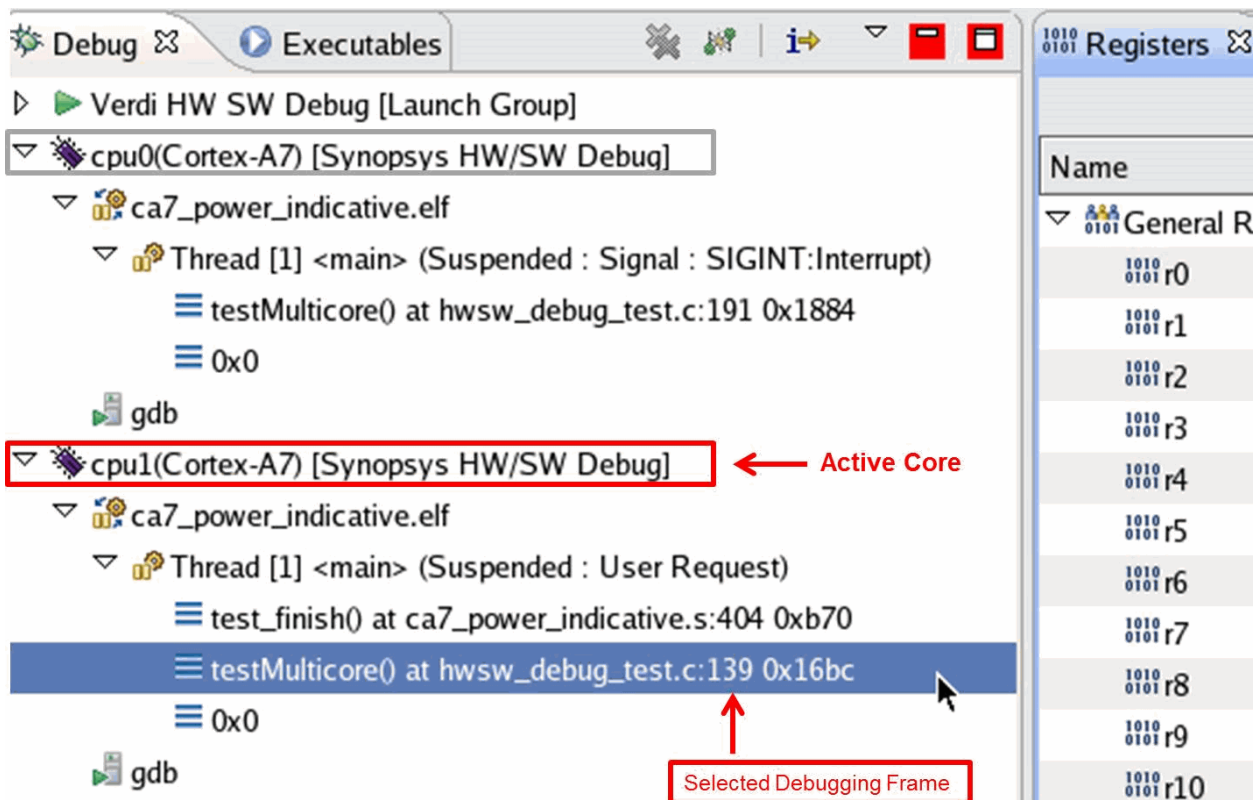
Figure 29 Multi-Core Debug View



Active Core and Switching to Different Active Core

When debugging a multi-core design, at any given moment, exactly one core is the active core and only this core is able to receive the debugging commands issued by the user. As shown in Figure 30, the active core is the one that has a debugging window selected in the *Debug view*.

Figure 30 Active Core



The currently active core is the one that provides necessary information for the other views in *Eclipse*, such as, *Registers view*, *Memory view*, *Variables view* and so on. In other words, these views display the corresponding information related only to the currently active core, except when the given view is *pinned* to a given core.

Selecting a debug window from another core within the *Debug view* can simply switch to a different current view. Immediately after switching the active core, all views in *Eclipse* that are not *pinned* to a specific core get updated with the information coming from the new active core.

Thread Local Breakpoint Checking Mode

Usually, breakpoints are active on all cores for which a matching source line exists. Whichever core hits a breakpoint first initiates the interrupt. When a breakpoint is hit on a core while stepping in another core, the step is interrupted and the focus changes to the core where the breakpoint is hit. To solely focus on one core, click the  icon on the *debug* toolbar of the *debug* view to enable thread local breakpoint checking.

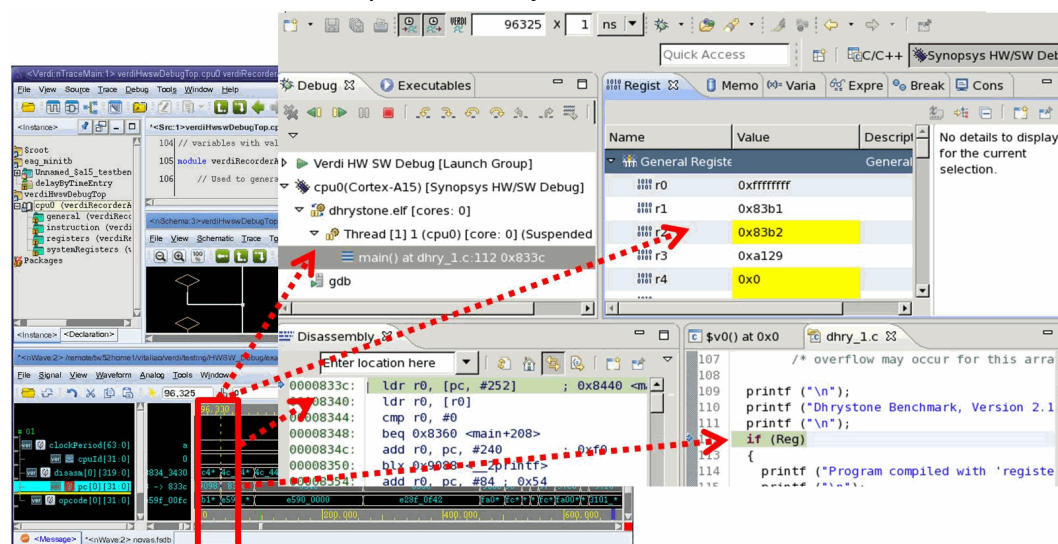
After the *thread local breakpoint checking* mode is enabled, only breakpoints on the active (currently focused) core are checked if they are hit. This also means that when stepping in one core the focus never changes to another core.

Synchronization With Hardware Debug

The views of *Verdi* platform and *Eclipse* are synchronized regarding time. When the time cursor in the *nWave* window is changed, the information displayed in the views of *Eclipse* is also updated. When the *Eclipse* view is changed by stepping or running to a breakpoint, the *Verdi* views are also updated to the same time. Breakpoints do not have an impact on this operation.

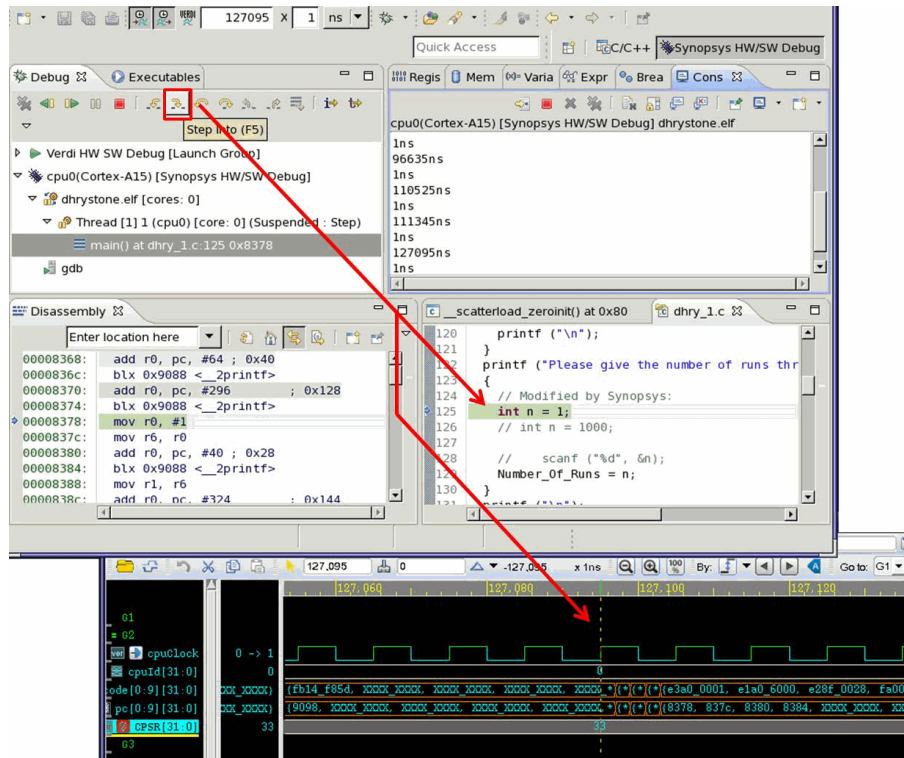
As demonstrated in [Figure 31](#), the called function is marked in the *Debug* view and the contents in the *Registers* view are also flushed for the current cursor location. The execution lines in *Disassembly* and *Source Code* view are also marked.

Figure 31 Set Cursor and Eclipse Views Synchronized



The synchronization is also done in the alternate direction. As soon as the current execution location is changed in *Eclipse* (after some **run** command), the time cursor in the *Verdi* platform is updated to reflect the simulation time where the current line of the C/Assembly source code is executed in the simulation (refer to [Figure 32](#)). When the value in the *Time Value* window of the *Eclipse* toolbar is changed, the *Verdi* windows also get updated to the same time.

Figure 32 Step Into Source Code and nWave Synchronized

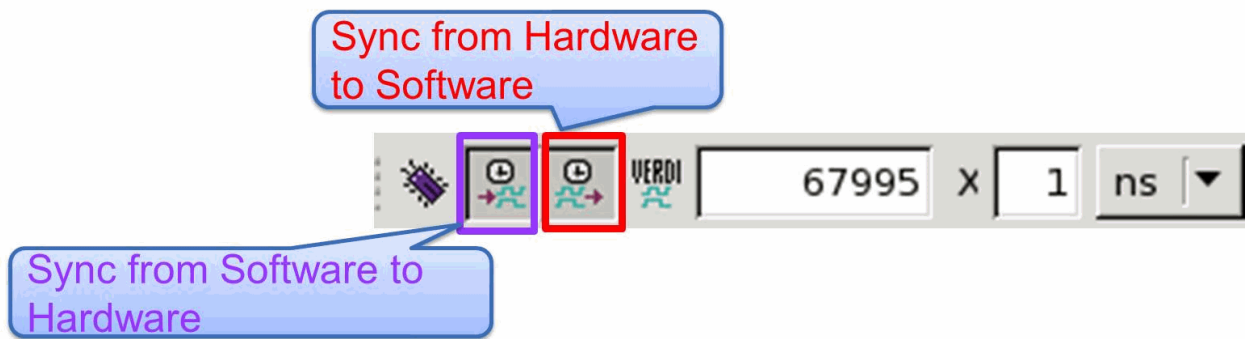


As demonstrated in [Figure 33](#), the synchronization direction can be specified on the toolbar:

- Click the  icon to enable synchronization from software to hardware.
- Click the  icon to enable synchronization from hardware to software.

By default, synchronization in both directions is enabled.

Figure 33 Synchronization Directions



4

Performance Profiling

Profiling allows you to measure where your program spent its runtime and allows you to learn which functions called which other functions during execution. This information allows you to trace and correct the parts of your program which are slower than you expected.

The HW/SW Debug solution allows you to measure the runtime performance of the software (SW) running on an embedded processor core. You can use the `gprof` option as an argument to the HW/SW converter to generate profiler reports detailing where time was spent by an embedded software application during execution.

The performance profiler report lists the following information for a function:

- Percentage of time spent in that function
- Average time spent in that function per call
- Which functions called it, and how often
- How often was the function called
- Which functions it called

This information allows you to quickly figure out the following:

- Portions of your code that cause slowdown
- Portions of your code that you should spend effort to optimize runtime performance

This feature allows you to get performance-related information without modifying the application code.

The following sections describe this feature in detail:

- [Generating Performance Profiler Reports](#)
- [Generating Assembly Listing Annotated with Execution Times](#)
- [Limitations](#)

Generating Performance Profiler Reports

Use the `hwsd_debug_convert` command along with the `gprof` option in the command line to generate profiling information. Following is the syntax:

```
hwsd_debug_convert -load_design design.xml gprof -o hwsd.fsd
```

Where, the *design.xml* file contains the configuration of the log file names, ELF file names, and CPU types.

Note:

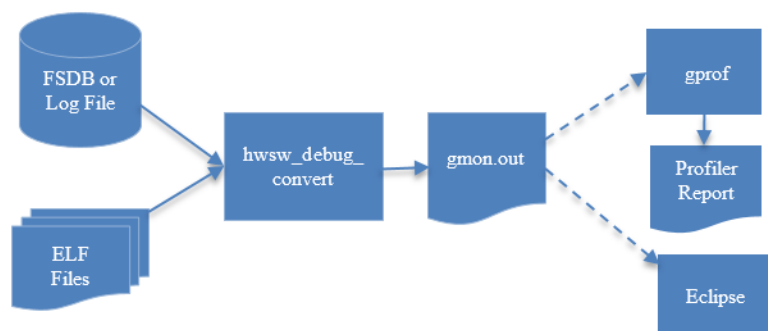
If `hwsd.fsd` file already exists, use the following syntax to generate profiling information: `hwsd_debug_convert -i hwsd.fsd gprof`

The `hwsd_debug_convert` command processes information in the CPU log file or an existing `hwsd.fsd` file and writes the profiling information into a profiler file with the format filename `.gmon.out` (for example, `cluster1.gmon.out`).

If the design contains multiple clusters, a separate profiler file is generated for each cluster. The `hwsd_debug_convert` command automatically selects distinct names, such as `cluster0.gmon.out`, `cluster1.gmon.out`, and so on.

You can view the profiling information in Eclipse or generate the profiler report in text format using the `gprof` tool, as shown in the following figure:

Figure 34 Viewing Performance Profiler Reports



Generating Profiler Reports in Text Format

You must use the `gprof` tool, as shown in [Figure 34](#) to generate the profiler report in text format. Following is the syntax:

```
gprof <elf_file> filename.gmon.out > filename.log
```

Where, `filename.log` is the text file to which the profiling information is written.

Example:

```
gprof ./test_sw/build32/hwsw_debug_test.elf cluster.gmon.out > report.log
```

Following is the sample gprof profile output in text format:

```
Flat profile:
Each sample counts as 1e-09 seconds.
% cumulative self self total
time seconds seconds calls ms/call ms/call name
9.65 0.00 0.00 101 0.00 0.00 _malloc_r
9.29 0.00 0.00 18 0.01 0.01 _write
8.90 0.00 0.00 80 0.00 0.00 memmove
8.58 0.00 0.00 1 0.15 0.41 corelTest
7.72 0.00 0.00 4 0.03 0.03 memset
6.68 0.00 0.00 18 0.01 0.04 _vfprintf_r
6.06 0.00 0.00 32 0.00 0.00 memchr
% the percentage of the total running time of the
time program used by this function.
cumulative a running sum of the number of seconds accounted
seconds for by this function and those listed above it.
self the number of seconds accounted for by this
seconds function alone. This is the major sort for this
listing.
Copyright (C) 2012-2014 Free Software Foundation, Inc.
Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.
Call graph (explanation follows)
granularity: each sample hit covers 2 byte(s) for 0.00% of 0.00 seconds
index % time self children called name
<spontaneous>
[1] 70.9 0.00 0.00 cpu0_start [1]
0.00 0.00 1/1 _hwsw_debug_start [2]
-----
0.00 0.00 1/1 cpu0_start [1]
[2] 70.8 0.00 0.00 1 _hwsw_debug_start [2]
0.00 0.00 1/1 main [3]
0.00 0.00 1/1 init_libc [32]
-----
This table describes the call tree of the program, and was sorted by
the total amount of time spent in each function and its children.
Each entry in this table consists of several lines. The line with the
index number at the left hand margin lists the current function.
The lines above it list the functions that called this function,
and the lines below it list the functions this one called.
This line lists:
index A unique number given to each element of the table.
Index numbers are sorted numerically.
The index number is printed next to every function name so
it is easier to look up where the function is in the table.
Copyright (C) 2012-2014 Free Software Foundation, Inc.
Copying and distribution of this file, with or without modification,
```

are permitted in any medium without royalty provided the copyright notice and this notice are preserved.

Index by function name

```
[51] __adddf3 [2] _hwsw_debug_start [3] main
[33] __aeabi_uldivmod [57] _init [22] malloc
[34] __gnu_uldivmod_helper [47] _localeconv_r [41] malloc_extend_top
[48] __libc_init_array [23] _malloc_r [28] memchr
[49] __malloc_lock [54] _sbrk_r [24] memmove
[40] __malloc_unlock [19] _svfprintf_r [25] memset
[46] __register_exitproc [6] _vfprintf_r [30] output_char
[16] __sflush_r [21] _write [5] printf
```

Viewing Profiler Reports in Eclipse

To view profiler reports in Eclipse, perform the following steps:

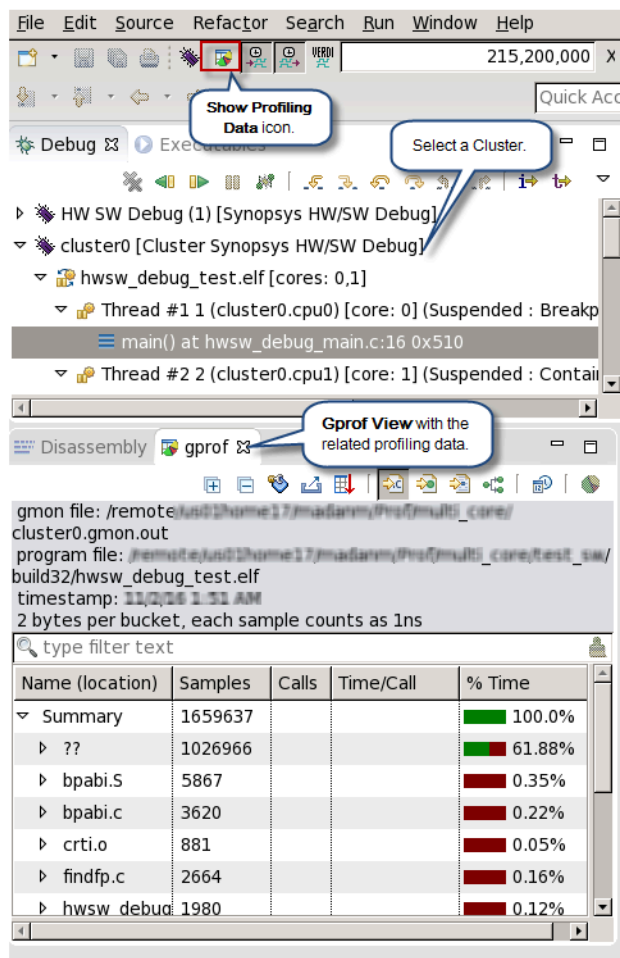
1. Select a Cluster in the *Debug View*, as shown in Figure 1-2.
2. Click the **Show Profiling Data** icon  in the *HW/SW Debug* toolbar.

Eclipse searches for the ELF and `gmon.out` files of the current selection in the *Debug View*, and if both are found, it automatically opens the *Gprof View* with the related profiling data, as shown in [Figure 35](#). Eclipse generates single profiler report for each cluster, if you select multiple clusters in the *Debug View*, Eclipse displays profiling information for all of them.

Chapter 4: Performance Profiling

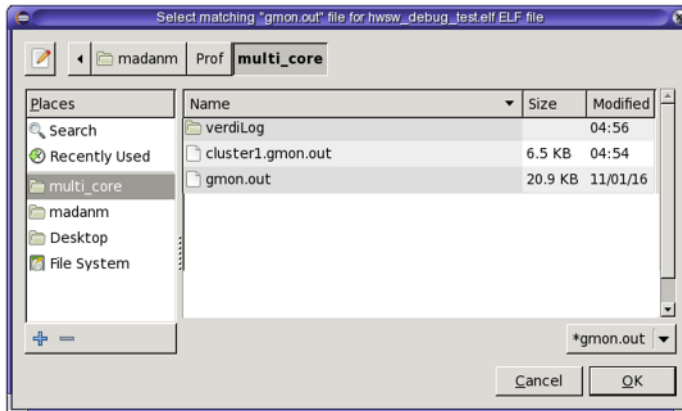
Generating Performance Profiler Reports

Figure 35 Viewing Profiler Information in Eclipse



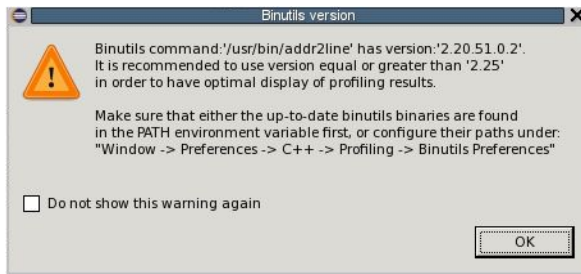
If Eclipse is not able to locate any or both of the ELF and `gmon.out` files, it displays the *Gmon File Viewer* dialog box, as shown in the following figure, which allows you to manually specify those files.

Figure 36 Gmon File Viewer Dialog Box



After the files are specified, the version of the Binutils is checked to see if it meets the minimum recommendations before opening the *Gprof View*. If the version of the Binutils does not meet the minimum recommended version, a warning dialog box is displayed as follows:

Figure 37 Binutils Version Dialog Box



Click the **OK** button in the *Binutils Version* warning dialog box to open the *Gprof View*.

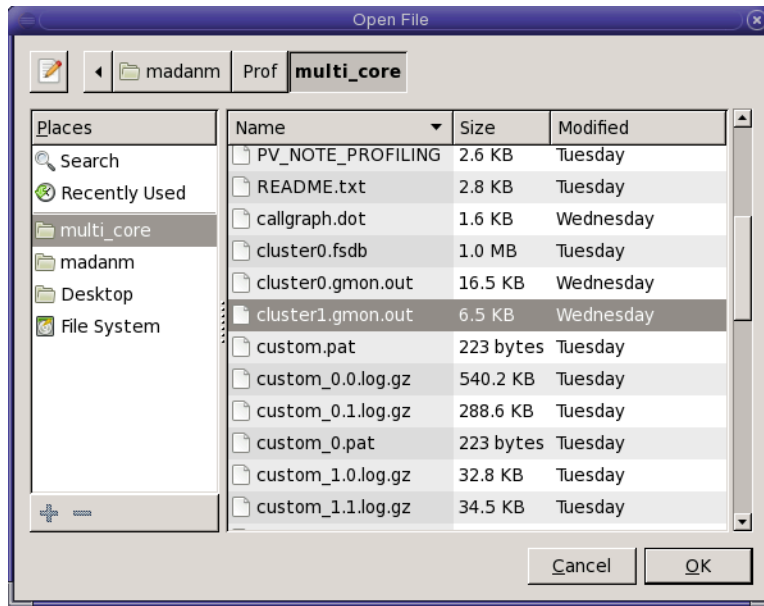
Note:

To resolve the Binutils warning message, make sure that the Binutils are picked up from the QSC compliant tool chain.

Alternatively, you can also perform the following steps to open the *Gprof View*:

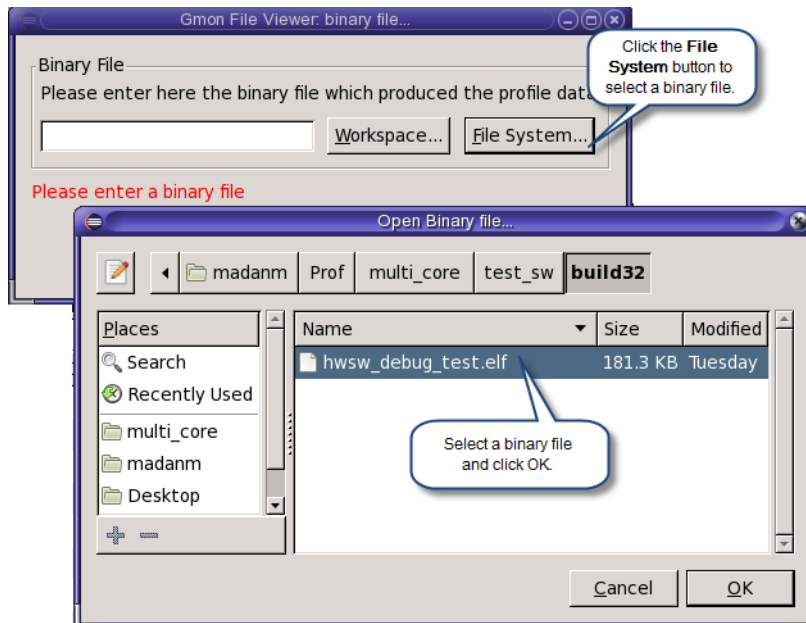
1. Click the **File > Open File** command.
2. Select a `gmon.out` file in the *Open File* dialog box, as shown in the following figure, and click the **OK** button.

Figure 38 Open File Dialog Box



3. Click the **File System** button in the *Gmon File Viewer* dialog box as shown in [Figure 39](#).

Figure 39 Selecting a Binary File

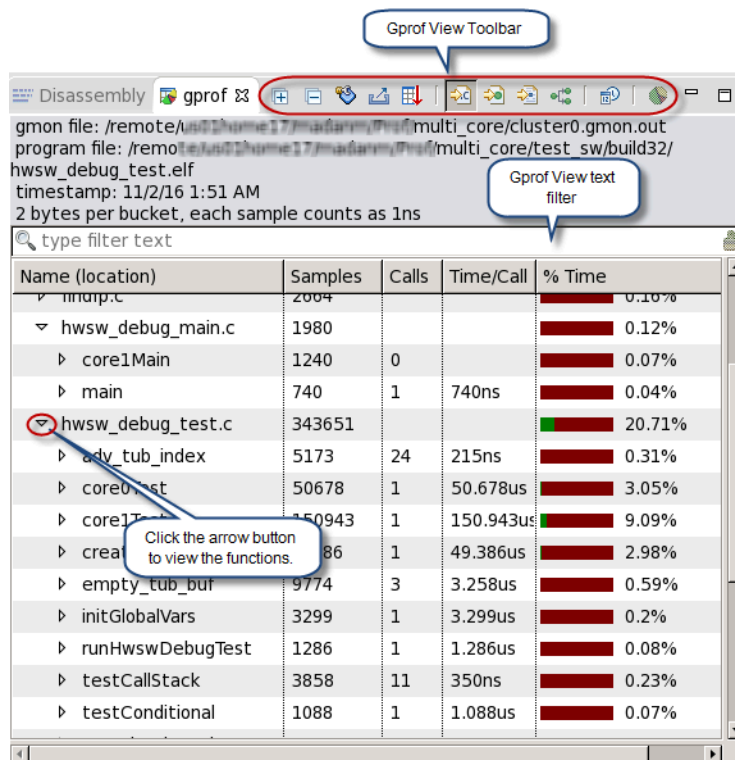


4. Select a binary file in the *Open Binary File* dialog box and click the **OK** button.
5. Click the **OK** button in the *Gmon File Viewer* dialog box to open the *Gprof View* with the related profiling data.

Gprof View

Figure 40 shows the *Gprof View*. It displays profiler information in tabular form and contains various icons to sort the profiler information.

Figure 40 *Gprof View*



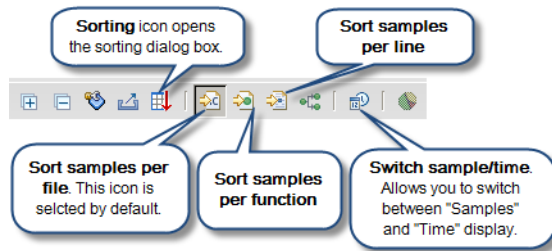
The *Gprof View* displays the profiler information described in the following columns:

Column Name	Description
Name (location)	Displays the file and function names
Samples	Displays the sample count
Calls	Displays the number of times this function was invoked, if this function is profiled, else blank

Time/Call	Displays the average time spent in this function per call, if this function is profiled, else blank
%Time	Displays the percentage of the total running time of the program used by this function

The *Gprof View* toolbar contains various icons, as shown in [Figure 41](#) to sort profiler information.

Figure 41 *Gprof View Toolbar*



Generating Assembly Listing Annotated with Execution Times

You can also create profiling information by creating an assembly listing annotated with the execution times.

To create an assembly listing annotated with the execution times, you must first create a disassembly listing. This depends on the tool you use:

- For `gcc/binutils`, use the `objdump -d <ELF_FILE>` command
- For `clang`, use the `llvm-objdump -d <ELF_FILE>` command
- For `RVDS/ARMCC`, use the `fromelf -c <ELF_FILE>` command

To create a disassembly listing, the output must be written into a `.dis` file (for example, `myapp.dis`). You can then create an annotated disassembly listing using the following command:

```
hws_debug_prof -d myapp.dis -g filename.gmon.out
```

For example, to create the disassembly file using `objdump`, execute the following commands:

```
% objdump -d ./test_sw/test.elf > myapp.dis
% hws_debug_prof -d myapp.dis -g file.gmon.out -o test.dis.log
```

Where, `myapp.dis` is the disassembly file and `test.dis.log` is the annotated disassembly file.

Following is the sample report of the `myapp.dis` disassembly file:

```
./test_sw/build64/hwsw_debug_test64.elf: file format elf64-littleaarch64
Disassembly of section .text:
0000000000000000 <bootcode>:
0: aalf03e0 mov x0, xzr
4: aalf03e1 mov x1, xzr
8: aalf03e2 mov x2, xzr
c: aalf03e3 mov x3, xzr
10: aalf03e4 mov x4, xzr
14: aalf03e5 mov x5, xzr
18: aalf03e6 mov x6, xzr
1c: aalf03e7 mov x7, xzr
20: aalf03e8 mov x8, xzr
0000000000000164 <ttb_setup>:
164: 100bf4e0 .word 0x100bf4e0
168: d51e2000 .word 0xd51e2000
16c: 18000301 .word 0x18000301
170: d51e2041 .word 0xd51e2041
174: 580af461 .word 0x580af461
178: d51ea201 .word 0xd51ea201
00000000000001c0 <wfi_loop>:
1c0: d503207f .word 0xd503207f
1c4: 17ffffff .word 0x17ffffff
00000000000001c8 <cll_cpu_start>:
1c8: 14000a1e .word 0x14000a1e
1cc: 80803520 .word 0x80803520
...
0000000000000800 <vector_table>:
800: d503201f nop
804: d2a26000 mov x0, #0x13000000 // #318767104
808: 10000061 adr x1, 814 <NGLOBAL+0x14>
80c: 940001fd bl 1000 <print>
810: 14000201 b 1014 <terminate>
814: 78656e55 .word 0x78656e55
...
0000000000000880 <curr_el_sp0_irq>:
880: d503201f nop
884: d2a26000 mov x0, #0x13000000 // #318767104
888: 10000061 adr x1, 894 <curr_el_sp0_irq+0x14>
88c: 940001dd bl 1000 <print>
890: 140001e1 b 1014 <terminate>
```

Following is the sample report of the `test.dis.log` annotated disassembly file:

```
./test_sw/build64/hwsw_debug_test64.elf: file format elf64-littleaarch64
Disassembly of section .text:
0000000000000000 <bootcode>
[      26625ns]    0: aalf03e0 mov x0, xzr
[      28680ns]    4: aalf03e1 mov x1, xzr
```

```

[          26625ns]      8: aa1f03e2 mov x2, xzr
[          26625ns]      c: aa1f03e3 mov x3, xzr

0000000000000164 <ttb_setup>:
[          30720ns]    164: 100bf4e0 .word 0x100bf4e0
[          61440ns]    168: d51e2000 .word 0xd51e2000
[          30723ns]    16c: 18000301 .word 0x18000301
[          22528ns]    170: d51e2041 .word 0xd51e2041
[          26625ns]    174: 580af461 .word 0x580af461
[          26624ns]    178: d51ea201 .word 0xd51ea201

00000000000001c0 <wfi_loop>:
[           0ns]     1c0: d503207f .word 0xd503207f
[          120ns]     1c4: 17ffffff .word 0x17ffffff

00000000000001c8 <cll_cpu_start>:
[          120ns]     1c8: 14000a1e .word 0x14000a1e
[           0ns]     1cc: 80803520 .word 0x80803520

...

0000000000000800 <vector_table>:
[           0ns]     800: d503201f nop
[          46080ns]    808: 10000061 adr x1, 814 <NGLOBAL+0x14>
[          30721ns]    80c: 940001fd bl 1000 <print>

...

0000000000000880 <curr_el_sp0_irq>:
[           0ns]     880: d503201f nop
[          29700ns]    888: 10000061 adr x1, 894 <curr_el_sp0_irq+0x14>
[          46080ns]    88c: 940001dd bl 1000 <print>
[          30720ns]    890: 140001e1 bl 1014 <terminate>

...

0000000000000900 <curr_el_sp0_fiq>:
[          720ns]     900: d503201f nop
[          120ns]     904: d2a26000 mov x0, #0x13000000 // #318767104
[           0ns]     908: 10000061 adr x1, 914 <curr_el_sp0_fiq+0x14>
[           0ns]     90c: 940001bd bl 1000 <print>
[           0ns]     910: 140001c1 b 1014 <terminate>
[          120ns]     914: 78656e55 .word 0x78656e55

...

0000000000000980 <curr_el_sp0_serror>:
[          120ns]     980: d503201f nop
[          120ns]     98c: 9400019d bl 1000 <print>
[          240ns]     990: 140001a1 b 1014 <terminate>
[          240ns]     994: 78656e55 .word 0x78656e55
[          240ns]     998: 74636570 .word 0x74636570

...

```

```
00000000000000a00 <curr_el_spx_sync>:
[          1ns]      a00: d503201f nop
[          0ns]      a08: 10000041 adr x1, a10
<curr_el_spx_sync+0x10>
[      28515ns]      a0c: 14000182 b 1014 <terminate>
[          0ns]      a10: 78656e55 .word 0x78656e55
[        120ns]      a14: 74636570 .word 0x74636570
[        120ns]      a18: 65206465 .word 0x65206465
[        360ns]      a1c: 70656378 .word 0x70656378

...
```

Limitations

Following are the limitations of this feature:

- If you want to view line-based profiling information, make sure that debug information (which provides information about address to source line mapping) is enabled when compiling your application. Note that this does not contradict the requirement to support optimized code, as most of the compilers support generating debug information when generating optimized code.
- The time consumed by the assembly instructions is calculated based on the data of the instruction log. This time might be inaccurate due to certain effects, depending on the complexity of the architecture.

5

Coverage Support

The following sections describe HW/SW Coverage support in detail:

- [Line Coverage Support](#)
- [Coverage Exclusion Support](#)

Line Coverage Support

Verdi HW/SW Debug supports line coverage for the C/C++ or Assembly application code which is run on an embedded processor core part of a design. This feature generates metrics for total coverage of lines that tell you how much of your code is actually executed by the tests. Verdi HW/SW Debug uses the new `hwsd_debug_coverage` tool to generate line coverage information.

Line coverage shows which lines of code are exercised, and which ones are not, by the application tests during a simulation run. Line coverage also produces annotated listing that identify the lines of code that are not exercised.

Generating Coverage Information

To generate line coverage information using Verdi HW/SW Debug, the first step is to generate the `gmon.out` file using the `hwsd_debug_convert` tool. The `gmon.out` file contains information to compute coverage. You can then use the new `hwsd_debug_coverage` tool to generate line coverage information in the following formats based on the `gmon.out` file:

Viewing coverage data in Eclipse (See [“Viewing Line Coverage Information in Eclipse”](#))

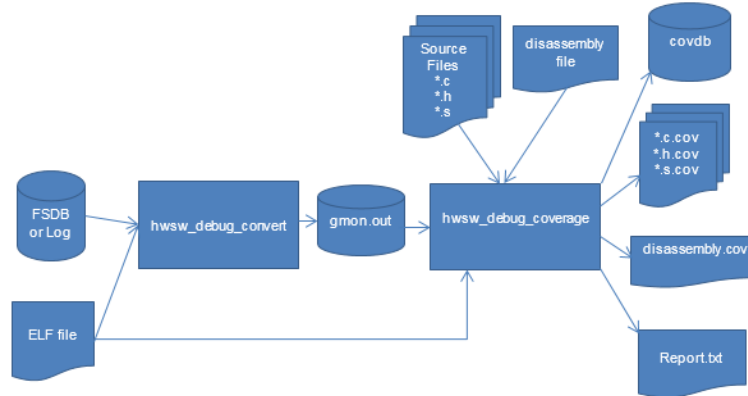
Text report (See [“Generating a Text Report”](#))

Annotated source files (See [“Generating Annotated Source Files with Line Coverage Information”](#))

Annotated disassembly files (See [“Generating Annotated Disassembly File with Line Coverage Information”](#))

The following figure shows the Verdi HW/SW Debug line coverage support:

Figure 42 Verdi HW/SW Debug Line Coverage Support



Use the `hws_debug_convert` command along with the `gprof` option in the command line to generate coverage information. Following is the syntax:

```
hws_debug_convert -load_design design.xml gprof -o hws.fldb
```

Where, the `design.xml` file contains the configuration of the log file names, ELF file names, and CPU types.

Note:

If the `hws.fldb` file already exists, use the following syntax to generate coverage information: `hws_debug_convert -i hws.fldb gprof`

The `hws_debug_convert` command processes information in the CPU log file or an existing `hws.fldb` file and writes the coverage information into a file with the format `filename.gmon.out` (for example, `cluster1.gmon.out`). The `gmon.out` file contains information to compute coverage.

Options Supported by the `hws_debug_coverage` Tool

The following table lists the options supported by the `hws_debug_coverage` tool:

Option	Description
<code>-l, -elf_file FILE</code>	The ELF file for which the coverage information should be generated.

Option	Description
<code>-g, -gmon_files FILE</code>	The input data file produced with the <code>gprof</code> option of the <code>hws_debug_convert</code> tool containing profiling and coverage information. If there are multiple <code>gmon.out</code> files (for multiple tests), you must separate them with commas. The results are the total coverage with the sum of the coverage from the individual files. The option defaults to <code>gmon.out</code> if no <code>gmon</code> files or file-lists are specified.
<code>-f, -gmon_list FILE</code>	A file with a list of <code>gmon</code> files (one file per line) produced with the <code>gprof</code> option of the <code>hws_debug_convert</code> tool containing profiling and coverage information. The results of all <code>gmon</code> files are merged (summed up). You can also provide multiple <code>gmon</code> file lists separated with commas.
<code>-d, -disassembly FILE</code>	Annotates the given disassembly file.
<code>-o, -annotated_disassembly FILE</code>	The output file that contains the content of the disassembly file annotated with the coverage information. Using "-" as filename writes the output on the STDOUT. Defaults to the name of the disassembly source file extended with ".cov".
<code>-report [FILE]</code>	Generates coverage report into the given file. If no filename is given, writes coverage report to the "coverage_report.txt" file.
<code>-annotate_sources DIR</code>	Annotates source files with coverage information in gcov format. If no output directory is given, a directory named "src_coverage" is created.
<code>-add_source_path DIR</code>	Adds one or multiple paths to search source files in. Searches in the sub-hierarchy given. Multiple paths must be separated by ":".
<code>-covdb FILE</code>	Changes the name of the file generated with coverage information for displaying in Eclipse. The default name is based on the name of the <code>gmon.out</code> file.

Option	Description
<code>-lcov_capture FILE</code>	Generates coverage information in the format compatible to <code>lcov .info</code> file. The default name is based on the name of the <code>gmon.out</code> file.
<code>-no_inlined_calls</code>	Does not add coverage for the lines containing inlined function calls . That is, inlined calls are displayed without code (there is no 'call' instruction due to inlining). The default is to generate additional coverage for inlined calls, making inlined calls appear like unoptimized calls if the DWARF debug information contains the necessary information.
<code>-parse_sources</code>	Some compilers fail to produce proper debug information for inlined functions. Still, the debug line information may reference source lines without function information. This can cause difficulties when computing coverage since inlined code cannot be correctly handled. The tool as a fallback offers to parse the C files in question, and add any new functions found to the list of known functions. .
<code>-show_dead_functions</code>	Normally functions containing only dead code (usually these are not called from anywhere) are suppressed. Use this option to still show them in the function tables.
<code>-exclusion_file FILE</code>	Adds a text file containing exclusion patterns which are excluded from coverage statistics. For information on the syntax of the exclusion file , perform <code>-help exclusion</code> .
<code>-mixed</code>	Displays annotated source code mixed with disassembly.
<code>-version</code>	Prints the HW/SW Debug version and license terms.
<code>-v, -verbose</code>	Turn on verbose output.
<code>-h, -help</code>	Prints help text.
<code>-vc_lic_wait</code>	Waits for a VC-DEBUG-RUNTIME license to become available. Default is to fall back to a LT license.

Option	Description
<code>-licdebug</code>	Reports license diagnostic information to the console. You can use this option for license debugging only.

Merging Coverage Data

To collect the coverage data for multiple tests, perform the following steps:

1. Run the simulation/emulation with the first test
2. Use the `hws_debug_convert` tool to generate a `gmon.out` file for the first test. For example, `test1/gmon.out`
3. Run the simulation/emulation with the next test
4. Use the `hws_debug_convert` tool to generate a `gmon.out` file for the next test. For example, `test2/gmon.out`
5. Repeat the last two steps (3 and 4) for each test in the test set.
6. Provide a comma separated list of all `gmon` files when running the coverage tool to generate the desired merged coverage information for all tests.

```
hws_debug_coverage -gmon_file test1/gmon.out,test2/gmon.out
```

Note:

The application ELF file must be the same for all runs. This is checked by the coverage tool.

When running an application on a multi-core processor, code executed on any of the cores is added to the coverage. This is equivalent to merging the coverage data calculated separately for each of the cores.

Requirements on ELF File

To generate annotated source, the ELF binary must contain debug information which allows to map code addresses back to source lines.

Optimizations might optimize certain code lines. The optimized code lines do not appear as covered/uncovered, just like other lines which did not result in any code, for example, comment or empty lines.

If there is no debug information, you still have the option to generate a disassembly based coverage report.

Viewing Line Coverage Information in Eclipse

The `.covdb` file containing coverage information is generated by the `hws_debug_coverage` tool. This file is meant to be read and displayed by the HW/SW Debug Coverage Eclipse plug-in.

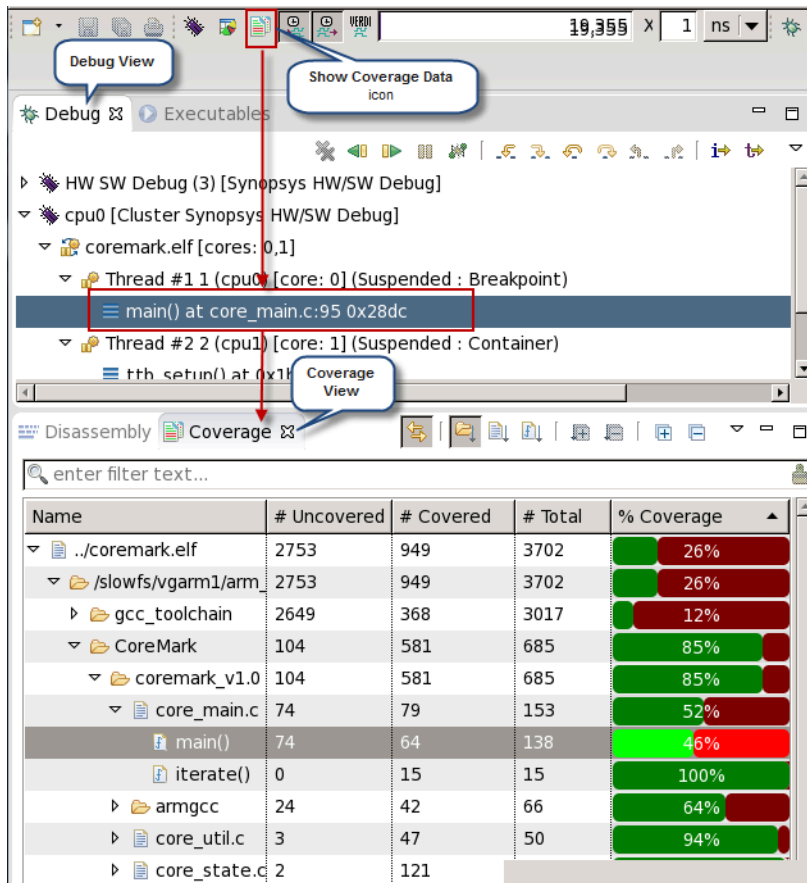
Following is the syntax to generate the `.covdb` file containing coverage information:

```
hws_debug_coverage -l test.elf -covdb hws.covdb  
hws_debug -ssf hws.fsdb &
```

Browsing Coverage Data at the File and Function Level in Eclipse

To view coverage data, click **Show Coverage Data** icon  in the Eclipse GUI.

The current selection in the debug view is taken into account to determine the corresponding coverage database file and, if found, automatically opens the **Coverage View** in the Eclipse GUI with the related coverage information, as shown in the following figure:



Note:

If you select more than one item in the **Debug View**, the system tries to open the coverage information for all of them.

If the system is not able to determine the corresponding coverage database file, a **File Open** dialog box is displayed to manually specify the file.

The **Coverage View** displays a hierarchical view showing coverage for source files and functions. The **Covered** and **Uncovered** columns display the covered and uncovered coverage points. The **Total** column displays the total number of coverage points, and the **Coverage** column displays the percentage of coverage, which is number of covered divided by the number of total coverage points.

The following table lists the functions available in the **Coverage View**:

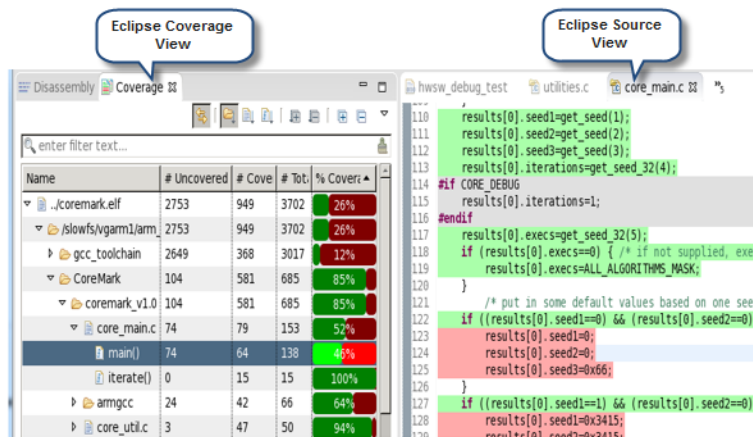
Function Name	Icon	Description
Link with the active editor		Selection in the Coverage View reflects the momentary selection in the active source viewer.

Function Name	Icon	Description
Switch to folder view		Displays the source hierarchy consisting of folders, source files, and functions as the leaf nodes.
Switch to file view		Displays a list of source files, each containing respective functions.
Switch to function view		Displays a list of all functions.
Filter		Displays results that only match the text specified in the filter text box.
Expand all items in the subtree		Expands the complete subtree of the selected folder item.
Collapse all items in the subtree		Collapses the complete subtree of the selected folder item.
Expand all items		Expands all the items in the Coverage View
Collapse all items		Collapses all the items in the Coverage View except the top two levels.

Viewing Covered/Uncovered Lines in Eclipse Source View

Double-click a function name in the Eclipse **Coverage View** to view its corresponding source in the Eclipse **Source View**, as shown in the following figure:

Figure 43 Covered/Uncovered Lines in Eclipse Source View



The Eclipse **Source View** displays the following:

- Covered statements containing code are displayed in green background color
- Uncovered statements containing code are displayed in red background color.
- Statements with not all of their underlying disassembly instructions executed, are marked as partially covered with yellow color.
- Lines which did not produce any code (for example, empty or comment lines) retains their background color.

Generating a Text Report

Following is the syntax to generate a text report with line coverage information:

```
hwsd_debug_coverage -elf_file test.elf -report report.txt
```

Example:

```
%> hwsd_debug_coverage -elf_file ./coremark.elf -g cluster0.gmon.out  
-report cluster0.log -v
```

The text report file contains the summary of the coverage information, including a list of all functions with their quota of covered lines, a list of all files with their quota, and an overall summary.

Example of Text Report Containing Coverage Percentage per Function

Following is the sample text report containing coverage percentage per function:

```
Synopsys HW/SW-Debug Code Coverage Report
```

```
=====
```

```
Created with: Verdi Release Verdi_M-2017.03-1   Built Apr 12 2017  
04:39:26
```

```
Creation time: Mon Apr 24 02:36:08 2017
```

```
General Information
```

```
=====
```

```
Number of runs combined: 1  
Total number of source lines: 3396  
Total number of covered lines: 949 ( 27.9%)  
Total number of uncovered lines: 2447 ( 72.1%)  
of which are partially covered: 3 ( 0.1% of uncovered)
```

```
Line Coverage by source file
```

```
=====
=====
/remote/us01home38/jefferry/arm_run_area/CoreMark/coremark_v1.0/
  51.6% ( 79/ 153) core_main.c
  94.0% ( 47/ 50) core_util.c
  98.4% ( 121/ 123) core_state.c
  99.5% ( 196/ 197) core_list_join.c
 100.0% ( 96/ 96) core_matrix.c
/remote/us01home38/jefferry/arm_run_area/CoreMark/coremark_v1.0/armgcc/
  63.0% ( 29/ 46) retarget-gcc.c
  65.0% ( 13/ 20) core_portme.c
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/gcc-linaro-4.9-201
4.06-1/libgcc/config/aarch64/
  0.0% ( 0/ 17) sfp-exceptions.c
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/gcc-linaro-4.9-201
4.06-1/libgcc/soft-fp/
  0.0% ( 0/ 8) truncdf2.c
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/newlib-linaro-2.1.
0-2014.02/libgloss/aarch64/
  0.0% ( 0/ 62) crt0.S
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/newlib-linaro-2.1.
0-2014.02/newlib/libc/locale/
  0.0% ( 0/ 28) locale.c
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/newlib-linaro-2.1.
0-2014.02/newlib/libc/machine/aarch64/
  0.0% ( 0/ 41) strcmp.S
  2.9% ( 5/ 175) memmove.S
 33.3% ( 29/ 87) memcpy.S
 51.5% ( 50/ 97) memset.S
100.0% ( 33/ 33) strlen.S
/remote/us01home38/jefferry/arm_run_area/gcc_toolchain/newlib-linaro-2.1.
0-2014.02/newlib/libc/misc/
  0.0% ( 0/ 6) fini.c
 55.6% ( 5/ 9) init.c

Number of source files:                    54
Number of completely covered source files: 7 (13.0%)
Number of partially covered source files:  22 (40.7%)
Number of fully uncovered source files:    25 (46.3%)
```

Coverage of functions, sorted by coverage:

```
=====
=====
0.0% ( 0b/ 8b) [0x000000000000001b8 ~ 0x000000000000001c0]
    wfi_loop
0.0% ( 0b/ 20b) [0x00000000000000800 ~ 0x00000000000000814]
    vector_table
0.0% ( 0b/ 20b) [0x00000000000000880 ~ 0x00000000000000894]
    curr_el_sp0_irq
0.0% ( 0b/ 20b) [0x00000000000000900 ~ 0x00000000000000914]
    curr_el_sp0_fiq
```

```

0.0% ( 0b/ 20b) [0x00000000000000980 ~ 0x00000000000000994]
    curr_el_sp0_serror
0.0% ( 0b/ 20b) [0x00000000000000a80 ~ 0x00000000000000a94]
    curr_el_spx_irq
0.0% ( 0b/ 20b) [0x00000000000000b00 ~ 0x00000000000000b14]
    curr_el_spx_fiq
0.0% ( 0b/ 20b) [0x00000000000000b80 ~ 0x00000000000000b94]
    curr_el_spx_serror
0.0% ( 0b/ 20b) [0x00000000000000c00 ~ 0x00000000000000c14]
    lower_el_aarch64_sync
0.0% ( 0b/ 20b) [0x00000000000000c80 ~ 0x00000000000000c94]
    lower_el_aarch64_irq
0.0% ( 0b/ 20b) [0x00000000000000d00 ~ 0x00000000000000d14]
    lower_el_aarch64_fiq
0.0% ( 0b/ 20b) [0x00000000000000f80 ~ 0x00000000000000f94]
    lower_el_aarch32_serror
100.0% ( 3/ 3) [0x000000000000005400 ~ 0x00000000000000541c]
    retarget-gcc.c:124~126 _arm_start()
100.0% ( 33/ 33) [0x000000000000005d40 ~ 0x000000000000005dc4]
    strlen.S:68~133 strlen()
100.0% ( 4/ 4) [0x000000000000005e00 ~ 0x000000000000005e2c] timesr.c:57~62
    _times_r()

Total number of functions:                205
Number of totally covered functions:       53 (25.85%)
Number of partially covered functions:    27 (13.17%)
Number of totally uncovered functions:    125 (60.98%)

```

Generating Annotated Source Files with Line Coverage Information

Following is the syntax to generate annotated source files with line coverage information:

```
hwswh_debug_coverage -elf_file file.elf -annotate_sources <new_dir>
```

Example:

```
%> hwswh_debug_coverage -elf_file ../coremark.elf -add_source_path
    test_sw/ -annotate_sources ann_path
```

You can generate a folder with annotated source files. For each source file an annotated copy of the source file is created under the respective path. The lines containing code are prefixed with the number of times they were executed (covered lines). The lines which are not executed (uncovered lines) are prefixed with "#####". The lines which did not result in any code are prefixed with "-". The lines which do not produce any code (for example, comment lines or variable declarations) are not counted as uncovered lines.

Example of Coverage Annotated Source File

Following is the sample coverage annotated source file:

```
-: 0:Source:core_main.c
-: 19:/* File: core_main.c
-: 20: This file contains the framework to acquire a block of memory,
   seed initial parameters, tun t he benchmark and report the results.
-: 21:*/
-: 22:#include "coremark.h"
-: 23:
-: 24:/* Function: iterate
-: 25: Run the benchmark for a specified number of iterations.
-: 26:
-: 27: Operation:
-: 28: For each type of benchmarked algorithm:
-: 29: a - Initialize the data block for the algorithm.
-: 30: b - Execute the algorithm N times.
-: 31:
-: 32: Returns:
-: 33: NULL.
-: 34:*/
-: 35:static ee_ul6 list_known_crc[] =
   {(ee_ul6)0xd4b0, (ee_ul6)0x3340, (ee_ul6)0x6a79, (ee_ul6)0xe714, (ee_ul6)0xe
3c1};
-: 36:static ee_ul6 matrix_known_crc[] =
   {(ee_ul6)0xbe52, (ee_ul6)0x1199, (ee_ul6)0x5608, (ee_ul6)0x1fd7, (ee_ul6)0x0
747};
-: 37:static ee_ul6 state_known_crc[] =
   {(ee_ul6)0x5e47, (ee_ul6)0x39bf, (ee_ul6)0xe5a4, (ee_ul6)0x8e3a, (ee_ul6)0x8
d84};
1: 38:void *iterate(void *pres) {
-: 39: ee_u32 i;
-: 40: ee_ul6 crc;
1: 41: core_results *res=(core_results *)pres;
1: 42: ee_u32 iterations=res->iterations;
1: 43: res->crc=0;
1: 44: res->crclist=0;
1: 45: res->crcmatrix=0;
1: 46: res->crcstate=0;
-: 47:
1: 48: for (i=0; i<iterations; i++) {
1: 49: crc=core_bench_list(res,1);
1: 50: res->crc=crcul6(crc,res->crc);
1: 51: crc=core_bench_list(res,-1);
1: 52: res->crc=crcul6(crc,res->crc);
1: 53: if (i==0) res->crclist=res->crc;
-: 54: }
1: 55: return NULL;
1: 56:}
-: 57:
-: 58:#if (SEED_METHOD==SEED_ARG)
-: 59:ee_s32 get_seed_args(int i, int argc, char *argv[]);
```

```

-: 60:#define get_seed(x) (ee_s16)get_seed_args(x,argc,argv)
-: 61:#define get_seed_32(x) get_seed_args(x,argc,argv)
-: 82: 2 - second seed : Must be identical to first for iterations to be
    identical
-: 83: 3 - third seed : Any value, should be at least an order of
    magnitude less than the input size, but bigger then 32.
-: 84: 4 - Iterations : Special, if set to 0, iterations will be
    automatically determined such that the benchmark will run between 10 to
    100 secs
-: 85:
-: 86:*/
-: 87:
-: 88:#if MAIN_HAS_NOARGC
-: 89:MAIN_RETURN_TYPE main(void) {
-: 90: int argc=0;
-: 91: char *argv[1];
-: 92:#else
1: 93:MAIN_RETURN_TYPE main(int argc, char *argv[]) {
-: 94:#endif
1: 95: ee_u16 i,j=0,num_algorithms=0;
1: 96: ee_s16 known_id=-1,total_errors=0;
1: 97: ee_u16 seedcrc=0;
-: 98: CORE_TICKS total_time;
-: 99: core_results results[MULTITHREAD];
-: 100:#if (MEM_METHOD==MEM_STACK)
-: 101: ee_u8 stack_memblock[TOTAL_DATA_SIZE*MULTITHREAD];
-: 102:#endif
-: 103: /* first call any initializations needed */
1: 104: portable_init(&(results[0].port), &argc, argv);
-: 105: /* First some checks to make sure benchmark will run ok */
-: 106: if (sizeof(struct list_head_s)>128) {
-: 107: ee_printf("list_head structure too big for comparable data!\n");
-: 108: return MAIN_RETURN_VAL;
-: 109: }
1: 110: results[0].seed1=get_seed(1);
1: 111: results[0].seed2=get_seed(2);
1: 112: results[0].seed3=get_seed(3);
1: 113: results[0].iterations=get_seed_32(4);
-: 114:#if CORE_DEBUG
-: 115: results[0].iterations=1;
-: 116:#endif
1: 117: results[0].execs=get_seed_32(5);
1: 118: if (results[0].execs==0) { /* if not supplied, execute all
    algorithms */
1: 119: results[0].execs=ALL_ALGORITHMS_MASK;
-: 120: }
-: 121: /* put in some default values based on one seed only for easy
    testing */
1: 122: if ((results[0].seed1==0) && (results[0].seed2==0) &&
    (results[0].seed3==0)) { /* validation run */
#####: 123: results[0].seed1=0;
#####: 124: results[0].seed2=0;
#####: 125: results[0].seed3=0x66;

```

```
-: 126: }
1: 127: if ((results[0].seed1==1) && (results[0].seed2==0) &&
    (results[0].seed3==0)) { /* performance run */
#####: 128: results[0].seed1=0x3415;
#####: 129: results[0].seed2=0x3415;
#####: 130: results[0].seed3=0x66;
-: 131: }
-: 132: #if (MEM_METHOD==MEM_STATIC)
-: 133: results[0].memblock[0]=(void *)static_memblk;
-: 134: results[0].size=TOTAL_DATA_SIZE;
-: 135: results[0].err=0;
-: 136: #if (MULTITHREAD>1)
```

Generating Annotated Disassembly File with Line Coverage Information

To generate annotated disassembly file with line coverage information, you must first create a disassembly listing. This depends on the tool you use:

- For *gcc/binutils*, use the *objdump -d <ELF_FILE>* command
- For *clang*, use the *llvm-objdump -d <ELF_FILE>* command
- For *RVDS/ARMCC*, use the *fromelf -c <ELF_FILE>* command

To create a disassembly listing, the output must be written into a *.dis* file (for example, *myapp.dis*). You can then use the following syntax to generate annotated disassembly file with line coverage information:

```
hws_debug_coverage -elf_file file.elf -disassembly file.dis
-annotated_disassembly ann_file.dis
```

Example:

```
%> hws_debug_coverage -elf_file ../coremark.elf -disassembly myapp.dis
-annotated_disassembly after_myapp.dis
```

The lines containing assembly instruction are prefixed with the number of times they were executed (covered lines). The instructions which are not executed (uncovered lines) are prefixed with "#####". The lines which did not result in any code are prefixed with "-".

Example of Coverage Annotated Disassembly File

Following is the sample coverage annotated disassembly file:

```
-: 2: ../coremark.elf: file format elf64-littleaarch64
-: 3:
-: 4:
-: 5: Disassembly of section .text:
-: 6:
```

```

-: 7: 0000000000000000 <bootcode>:
1: 8: 0: aalf03e0 mov x0, xzr
1: 9: 4: aalf03e1 mov x1, xzr
1: 10: 8: aalf03e2 mov x2, xzr
1: 11: c: aalf03e3 mov x3, xzr
1: 12: 10: aalf03e4 mov x4, xzr
1: 13: 14: aalf03e5 mov x5, xzr
1: 14: 18: aalf03e6 mov x6, xzr
1: 15: 1c: aalf03e7 mov x7, xzr
1: 36: 70: aalf03fc mov x28, xzr
1: 37: 74: aalf03fd mov x29, xzr
1: 38: 78: aalf03fe mov x30, xzr
1: 39: 7c: 9100001f mov sp, x0
1: 40: 80: d5184100 msr sp_el0, x0
1: 41: 84: d51c4100 msr sp_el1, x0
1: 42: 88: d51e4100 msr sp_el2, x0
1: 43: 8c: d5184020 msr elr_el1, x0
1: 44: 90: d51c4020 msr elr_el2, x0
1: 45: 94: d51e4020 msr elr_el3, x0
1: 53: b4: d53800a2 mrs x2, mpidr_el1
1: 54: b8: 92401c42 and x2, x2, #0xff
1: 55: bc: d2808003 mov x3, #0x400 // #1024
1: 56: c0: 9b037c43 mul x3, x2, x3
1: 57: c4: cb030021 sub x1, x1, x3
1: 58: c8: 9100003f mov sp, x1
1: 59: cc: d5380400 mrs x0, id_aa64pfr0_el1
1: 60: d0: d3504c00 ubfx x0, x0, #16, #4
1: 61: d4: b5000480 cbnz x0, 164 <ttb_setup>
#####: 62: d8: d2a00601 mov x1, #0x300000 // #3145728
#####: 63: dc: d5181041 msr cpacr_el1, x1
#####: 64: e0: d5033fdf isb
#####: 65: e4: 9e6703e0 fmov d0, xzr
#####: 66: e8: 9e6703e1 fmov d1, xzr
#####: 67: ec: 9e6703e2 fmov d2, xzr
#####: 68: f0: 9e6703e3 fmov d3, xzr
#####: 69: f4: 9e6703e4 fmov d4, xzr
#####: 70: f8: 9e6703e5 fmov d5, xzr
#####: 71: fc: 9e6703e6 fmov d6, xzr
#####: 72: 100: 9e6703e7 fmov d7, xzr
#####: 73: 104: 9e6703e8 fmov d8, xzr
#####: 74: 108: 9e6703e9 fmov d9, xzr
#####: 75: 10c: 9e6703ea fmov d10, xzr
#####: 76: 110: 9e6703eb fmov d11, xzr
#####: 77: 114: 9e6703ec fmov d12, xzr
#####: 78: 118: 9e6703ed fmov d13, xzr
#####: 79: 11c: 9e6703ee fmov d14, xzr
#####: 80: 120: 9e6703ef fmov d15, xzr
#####: 93: 154: 9e6703fc fmov d28, xzr
#####: 94: 158: 9e6703fd fmov d29, xzr
#####: 95: 15c: 9e6703fe fmov d30, xzr
#####: 96: 160: 9e6703ff fmov d31, xzr
-: 97:
-: 98: 0000000000000164 <ttb_setup>:

```

```
1: 99: 164: 100af4e0 .word 0x100af4e0
1: 100: 168: d51e2000 .word 0xd51e2000
1: 101: 16c: 180002e1 .word 0x180002e1
1: 102: 170: d51e2041 .word 0xd51e2041
1: 103: 174: 5809f461 .word 0x5809f461
1: 104: 178: d51ea201 .word 0xd51ea201
1: 105: 17c: d53e1000 .word 0xd53e1000
```

Coverage Exclusion Support

Coverage data is collected over multiple tests and simulations. Analyzing the accumulated coverage results uncovers parts of a design that have not been exercised and validated. These uncovered parts are called coverage holes. Some coverage holes can be excluded, such as debug code, because part of the design functionality will not be used in valid scenarios.

Verdi HW/SW Debug supports specifying coverage exclusion with an exclusion file.

Specifying Exclusions

You can use the following command to specify exclusion (text only) patterns in a file:

```
% hws_debug_coverage -exclusion_file <FILE>
```

This command allows you to specify a text file containing exclusion patterns to be excluded from coverage statistics. For a description of the syntax of an exclusion file, use the `-help exclusion` command as follows:

```
% hws_debug_coverage -help exclusion
```

A coverage exclusion file (`FILE`) is a line based text file where each line describes one exclusion. The following forms of exclusions are supported:

- Any line starting with a `#` is a comment, and is ignored.
- The following pattern excludes all source files named `dhry_1.c`.
`dhry_1.c:`
- The following pattern excludes line 300 in all source files named.
`dhry_1.c: 300`
- The following pattern excludes the range of lines starting with 200 and ending with 220:
`dhry_2.c: 200-220`
- The following pattern excludes all coverage points in function `Proc_6` of file `dhry_2.c`:
`dhry_2.c: Proc_6`

- The following pattern excludes one line in function `Func_2` relative to the function header:

```
dhry_2.c: Func_2 +26
```

- The following pattern excludes range of lines in function `Func_2` relative to the function header:

```
dhry_2.c: Func_2 +32-34
```

- The following pattern excludes all functions named `Proc_5`, in all files:

```
: Proc_5
```

- The following pattern excludes all coverage points in `.c` files matching the name `dhry_*.c`:

```
dhry_*.c:
```

- The following pattern excludes all files in the sub-folder `drivers/other`:

```
drivers/other/*:
```

- The following pattern excludes any source file in the folder `drivers`, or any sub-folder:

```
drivers/:
```

6

Adding a Custom Core

This chapter describes the procedures to customize the Synopsys HW/SW Debug environment to work with your custom processor.

Customizing the HW/SW Debug environment involves the following three steps detailed in this chapter.

- Create a configuration file with the names of all CPU registers
- Create a pattern file to parse your CPU log file
- Validate your CPU Core in the HW/SW Debug environment

The following sections describe this feature in detail:

- [Prerequisites](#)
- [Creating the HW/SW Debug Configuration File](#)
- [Creating the Log-Pattern File](#)
- [Testing and Debugging](#)

Prerequisites

The following topics describe the prerequisites:

- [Software Development Tool \(Compiler\)](#)
- [Debugger \(GDB\)](#)
- [HDL Simulator](#)

Software Development Tool (Compiler)

The software compile tool for your processor must generate the executable file (or called application file) in the format acceptable by the GDB used for this integration. The application file is loaded into GDB during the HW/SW Debug session. The HW/SW Debug server loads the application content into its memory for viewing in the Eclipse environment.

Debugger (GDB)

A GDB (GNU Debugger, <http://sourceware.org/gdb>) retargeted for your processor architecture is required. It is recommended to use the latest version available for your CPU core, and the GDB version must be v7.0 or later. GDB must be able to connect to a target using the Remote Serial Protocol (RSP) over a TCP/IP connection (via the command `target remote :port`). Normally this should not be an issue, however, for some heavily customized versions of GDB, it might be.

HDL Simulator

The Verdi tool suite works with all simulation vendors. The `hwswh_debug_convert` application parses the CPU log file, so there is no dependency on the simulator used. The simulator must generate an FSDB file for any design signals displayed in Verdi.

Creating the HW/SW Debug Configuration File

The Synopsys HW/SW Debug requires a configuration file in XML format to specify the registers available for your custom processor. You can find the processor registers using the GDB debugger used for this integration. You can use the registers specified in the configuration file to map the registers specified in the FSDB file to the registers in the GDB.

The registers are mapped using the internal GDB register numbering order. To find the register names and numbers in the GDB, you can run the GDB executable with one of the following commands. You can use the `help internal` command to get help on the command format for your GDB.

```
% custom-gdb \ --ex "maintenance print remote-registers" \ --ex "quit" |& tee log
% custom-gdb \ --ex "maintenance print registers" \ --ex "quit" |& tee log
```

The above command prints the register information similar to the following:

```
Name Nr Rel Offset Size Type
zero 0 0 0 4 uint32_t
r1 1 1 4 4 uint32_t
r2 2 2 8 4 uint32_t
... more GPRs register
pc 32 32 128 4 uint32_t
status 33 33 132 4 uint32_t
estatus 34 34 136 4 uint32_t
sp 35 35 140 4 uint32_t
... more Control/Flags registers
```

From the above register names and the Rel and Nr columns, you can create the following configuration file with the syntax given below. A Perl script is available to translate the GDB log into the XML syntax.

```
% hws_w_reg_to_xml.pl archname logcore_name.xml
<?xml version="1.0" encoding="UTF-8"?>
<esw_debug_configuration>
<architecture name="MyCore">
<reg name="zero" bitsize="32" path="core_regs[0]"/>
<reg name="r1" bitsize="32" path="core_regs[1]"/>
... More GPRS info here ...
<reg name="pc" bitsize="32" path="pc" is_pc='1' />
<reg name="status" bitsize="32" path="ctl_regs[0]"/>
<reg name="estatus" bitsize="32" path="ctl_regs[1]"/>
... More Control/Flags register here ...
<mem name="mem" addrsz="28" path="memory_access"/>
</architecture>
</esw_debug_configuration>
```

The Verdi HW/SW Debug tool must know which register is the program counter. The Perl script tries to detect this (any register called pc). Check that this is correct. Edit if needed, so that you have one `is_pc='1'` tag, and it is attached to the PC of your core as follows:

```
<reg name="pc" bitsize="32" path="pc" is_pc='1' />
```

Creating the Log-Pattern File

The Verdi HW/SW-Debug tool requires information about each opcode executed during the test so that you can re-create the CPU state at any point in time for post-process debugging. During simulation, a CPU log file is typically created, and you must convert this text information into a binary FSDB file for use by Verdi HW/SW Debug.

A converter tool is included in the Verdi package to perform this conversion. The converter requires a log pattern file with regular expression syntax to allow easy parsing of your specific log file format.

The pattern file contains regular expressions and also some special tags with a `%TAGNAME%` syntax. The following tags are recognized:

Table 1 Supported Tags

Tag Name	Tag Type	Description
%TIME%	Decimal	Time, requires TIMEUNIT
%TIMEUNIT%	String : ns, ps, fs	Time unit
%TIMESTAMP%	%TIME% %TIMEUNIT%	Timestamp

Table 1 *Supported Tags (Continued)*

%CYCLE%	Decimal	Cycle count, %TIME% preferred
%PC%	Hex	Program Counter
%REGNAME%	String : R0, R1, X0, X1, and so on	Register Name
%REGDATA%	Hex (optional leading 0x)	Register Data/Value
%REGNUM%	Decimal	Register number, use with a base register, for example: %REGNUM+X0%
%RDWR%	String “RD” or “WR”	Read/Write, same as Load/Store
%LDST%	String “LD” or “ST”	Read/Write, same as Load/Store
%VADDR%	Hex (optional leading 0x)	Memory Address
%MEMDATA%	Hex (optional leading 0x)	Memory Data
%MEMSIZE%	Decimal	Memory Size in Bytes
%HEX%	Hex Digits (0-9, a-f, A-F)	Hex number, ignored
%DEC%	Decimal Digits (0-9)	Decimal number, ignored
%OPCODE%	Hex	Matches opcode
%OPCODE_SIZE%	Decimal	Matches bit size of opcode
%DISASM%	String	Matches disassembly
%INDEX%	Decimal	Matches instruction count

Standard regular expression syntax is also supported to match other text during parsing.

Table 2 *Expression Syntax*

Regular Expression	Description
--------------------	-------------

Table 2 Expression Syntax (Continued)

ABC	Matches “ABC”
[ABC]	Matches “A” or “B” or “C”
(ABC)?	Matches optional “ABC”
.*	Matches zero or more characters

Testing and Debugging

This final step validates the configuration file which you created earlier in the process. This step involves testing the register configuration file and log pattern file along with the GDB integration.

Validating the Converter Pattern File

The converter can be executed on a sample CPU log file, FSDB file, or generated text file. It is recommended to initially generate a text file output as this is easier to manually validate the results of a short test.

```
% hws_debug_convert \
    cpuType=MyProc \
    patternFile=MyProc.pat \
    configName=MyProc \
    exeFile=test.elf \
    gdbPath=custom_gdb \
    clockPeriod=1ns \
    -C `pwd` \
    -i cpu.log \
    -o output.txt
```

The Register definition file is specified by `configName`, and `hws_debug_convert` searches for `MyProc.xml` in the current directory (or the directory specified by the `-C` argument).

The `clockPeriod` is required if your CPU log has cycle counts. This parameter should be removed if you have ns or ps (or other time units) in the log file.

The output format is Verdi FSDB if the `.fsdb` extension is specified. Any other extension generates a text file that allows you to see the result of the conversion.

If any lines do not match a pattern from the log pattern file, then an error is issued. You can use these errors to debug the line that does not match any pattern.

If register names do not match, then review the `MyProc.xml` file to see if the register names are correct.

It is useful to view the original CPU log file and the generated `output.txt` file together and perform some basic manual checking for some program counter values, register values, and memory read/write transactions. Just a few of these lines should be checked to ensure that the data was parsed correctly. Note that a single pattern with “.” matches all lines but does not extract data from the CPU log.

Rerun the conversion, and specify `-o hwsf.fsdb` to generate an FSDB file for validation.

Invoking Verdi HW/SW Debug Tool

Start by invoking Verdi HW/SW debug from the eclipse side. It is easier to debug any simple issue with this invoke method. If required, invoke from the Verdi tool later.

```
% hwsf_debug -C `pwd` -ssf hwsf.fsdb
```

If the Eclipse environment does not invoke or fails to invoke GDB, you can troubleshoot as follows:

1. Register-Configuration File Not Found

- Specify the path to the directory containing the register configuration file

```
% hwsf_debug -C config_directory  
% hwsf_debug -config_path config_directory
```

- Check if the name of the `configName` parameter during conversion is correct. Verdi automatically adds “.xml” to the end of this name.

2. GDB not found

Invoke your `custom_gdb` from the `PATH` environment variable.

- Check if the `gdbPath=custom_gdb` name is correct during the `hwsf_debug_convert` step.
- Ensure that the `custom_gdb` is on your path:

```
% which custom_gdb
```

3. Other Errors

- Check the Verdi console for any error message and correct the problem
- Check for any GDB connection error message in the `$cwd/hwsfDebugLog/HwSwDebug.log`
- Check if the `.gdbinit` files under `$cwd` or `$HOME` exist, and if they are correct.

If the Eclipse and GDB tools do not invoke, contact Synopsys for further assistance.

Integration Test

In this test, each register is loaded with a specific value and various memory read/write statements are executed. This allows you to test the basic functionality that the Eclipse environment uses to show complex C data and other data types. If possible, run a short test that loads each register with a fixed value and convert the log to FSDB. Then step through the code in Eclipse one opcode at a time:

- Switch to instruction stepping mode (select the "i" icon in the Debug view)
- Check if the PC value is correct and the disassembly matches the CPU log file
- Check that the time displayed in Eclipse matches the time in the CPU log file
- Check if the register value updates after each step matches the disassembly
- Check if stepping over a store instruction updates the memory view

If you see behavior you do not understand, contact Synopsys for further assistance.

Appendix: Perl Script to Generate Register.XML File

```
#!/usr/local/bin/perl
# (C) Synopsys 2022
# Run GDB first, to get a list of registers
# Csh% gdb --ex "maint print remote-registers" --ex q |& tee reg.txt
# Then run this script to generate the XML file for your core
# Csh% ./gdb_reg_to_xml.pl archname reg.txt > core.xml
# archname is the name of the GDB architecture: arm, aarch64 etc

use strict;

my ($arch) = @ARGV;
if (not defined $arch) {
    print "Usage: hwsw_reg_to_xml.pl archname reg.txt > core.xml\n";
    exit;
}
# Shift the arch-name so we read the reg-file next
shift;
print "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
print "\n"; print "<esw_debug_configuration>\n";
print "  <architecture name=\"$arch\">\n";
print "  <feature name=\"org.gnu.gdb.$arch.core\">\n";
while (<>) {
    if (m/([a-zA-Z0-9_]+)\s+(\d+)\s+(\d+)\s+(\d+)\s+(\d+)/) {
        my ($name, $idx, $bytes) = ($1, $2, $3);
        my $bits = $bytes * 8;
        my $pc;
```

```
        # Look for extra tags to add
        my $tags = "";
        if ($name =~ m/^pc$/i) { $tags = "is_pc='1'"
type="\code_ptr"/>"; }
        if ($name =~ m/^lr$/i) { $tags = "type="\code_ptr"/>"; }
        if ($name =~ m/^sr$/i) { $tags = "type="\data_ptr"/>"; }

        # Print the line
        printf " <reg name=\"%s\" bitsize=\"%sbits\"
path=\"registers.\";
                print uc $name;
                print "\" $tags />\n";
    }
}
print " </feature>\n";
print " <mem name=\"mem\" addrsize=\"32\"
path=\"general.memoryAccess\"/>\n";
print " </architecture>\n";
print "</esw_debug_configuration>\n";
print "\n";
```

7

Demo

The following sections provide information about demo examples and describe the steps that must be performed to run these examples:

- [Demo Example 1](#)
 - [Demo Example 2](#)
-

Demo Example 1

This demo uses the *dhrystone* example located in the `$VERDI_HOME/demo/hwsw_debug/dhrystone` directory. This example demonstrates how to invoke *Eclipse* from the Verdi platform to debug the design.

Perform the following steps to run the demo:

1. Copy the test case to your local directory:

```
%> cp -r $VERDI_HOME/demo/hwsw_debug/dhrystone.
```

2. Import the design with the recorder module library and load the generated FSDB file to the Verdi platform:

```
%> gmake hwsw_debug
```

The following command displays the list of available targets and the description:

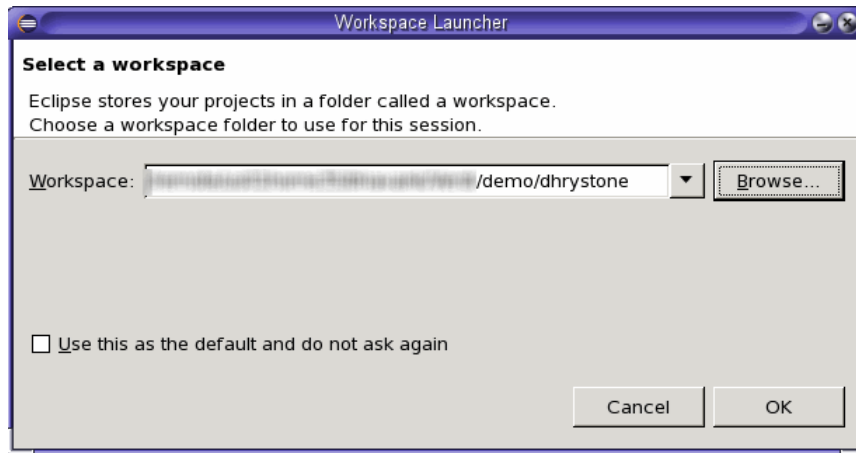
```
%> gmake help
```

3. Enter *Hardware Debug Mode*:

- In the *Welcome Page*, click the **Work Mode** icon.
- Select the **Hardware Debug Mode** option.
- Click the **Go to Work** button to enter *Hardware Debug Mode*.

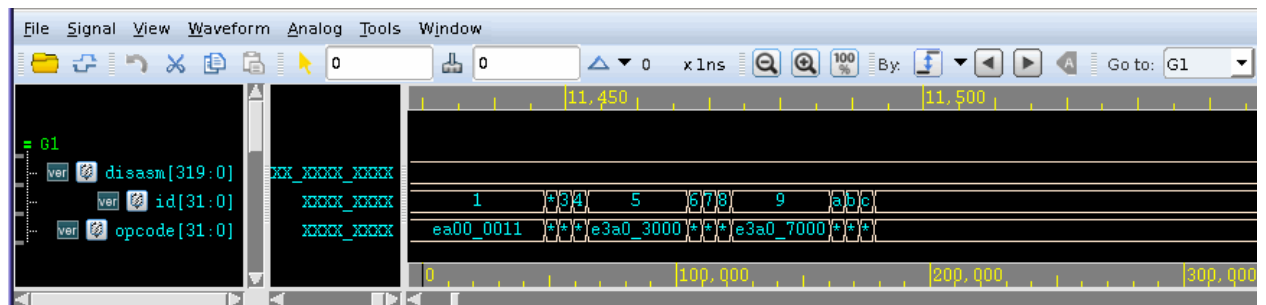
4. Invoke HW/SW Debug using the **Tools -> Invoke HW SW Debug** command. Click the **OK** button to confirm the design path field of the FSDB file. Then, specify a workspace to store the debug project the first time HW/SW Debug is invoked.

Figure 44 Workspace Launcher



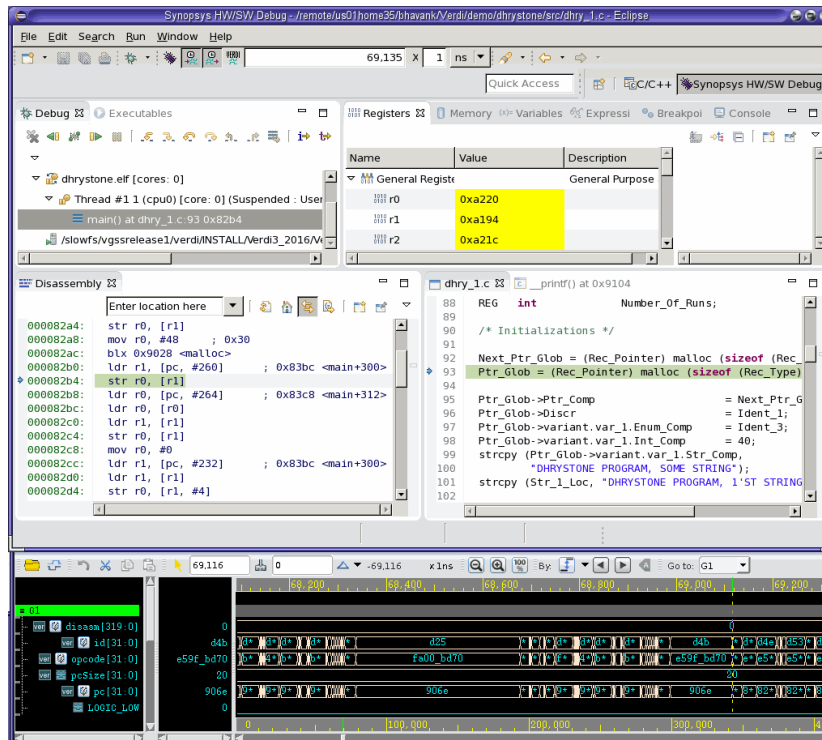
5. Click the **Yes** button in the *Confirm Perspective Switch* form to confirm the perspective switch and click the **Yes** button in the *Customize Preference* form to use the recommended value for preference.
6. Get the *opcode* and *pc* signals of the *verdiHwswDebugTop.cpu0.instruction* scope in the *nWave* window using the **Signal -> Get Signals** command to see the executed operation code and the program counter of the Cortex-A15 ARM core.

Figure 45 Get the CPU Signals



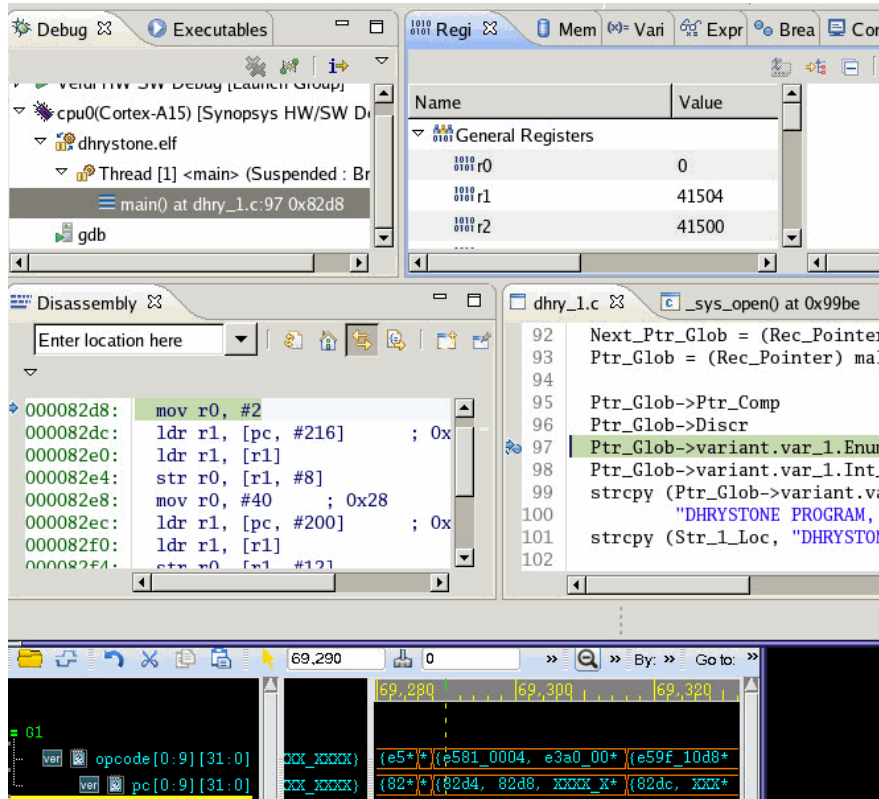
7. Place the cursor at time 69116ns in the waveform to see the debug line has changed in *Eclipse*. The code being executed is highlighted in both *Disassembly* and *Source Code* views. The call stack is also marked in the *Debug* view to specify the function being executed at 69116ns.

Figure 46 Get the CPU Signals in Eclipse



8. Set a breakpoint by double-clicking at line 97 in the *Source Code* view. Continue debugging using the **Resume** command to check whether the operation code is executed and to check the value of the program counter at the breakpoint.

Figure 47 Setting a Breakpoint



Demo Example 2

This demo uses the `multi_core` example located in the `$VERDI_HOME/demo/hwsw_debug/convert/multi_core` directory and demonstrates how to use `hwsw_debug_convert` functionality to convert a default log file format to a FSDB file which is used in *HW/SW Debug Solution*.

The design consists of two clusters, each with two cores and each core runs different tests. Cluster0 runs with 32 bit-elf and Cluster1 runs with 64 bit-elf files.

Perform the following steps to run the demo:

1. Copy the test case to your local directory:

```
%> cp -r $VERDI_HOME/demo/hws_debug/multi_core .
```

The following command displays more information about the demo content and the usage information:

```
%> gmake help
```

2. Convert the log files to FSDB file:

```
make convert
```

3. Invoke Verdi

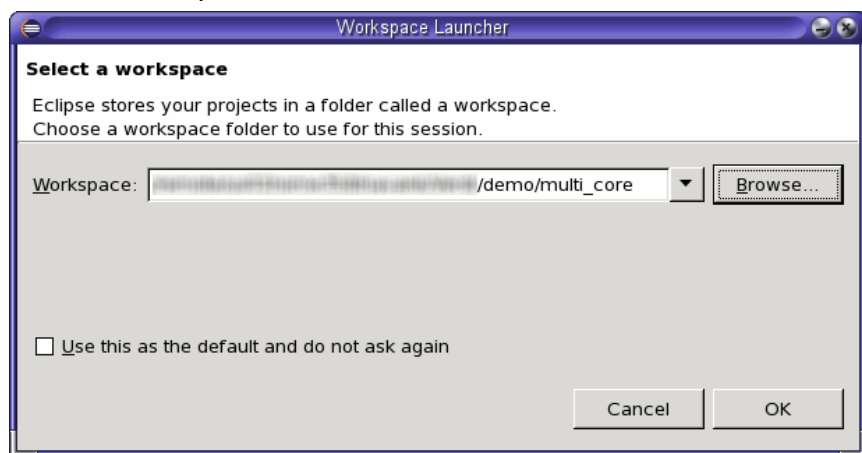
```
make verdi
```

4. Verdi GUI opens up. Enter *Hardware Debug Mode*:

- In the *Welcome Page*, click the **Work Mode** icon.
- Select the **Hardware Debug Mode** option.
- Click the **Go to Work** button to enter *Hardware Debug Mode*.

5. Invoke HW/SW Debug using the **Tools -> Invoke HW SW Debug** command. Click the **OK** button to confirm the design path field of the FSDB file. Then, specify a workspace to store the debug project the first time HW/SW Debug is invoked.

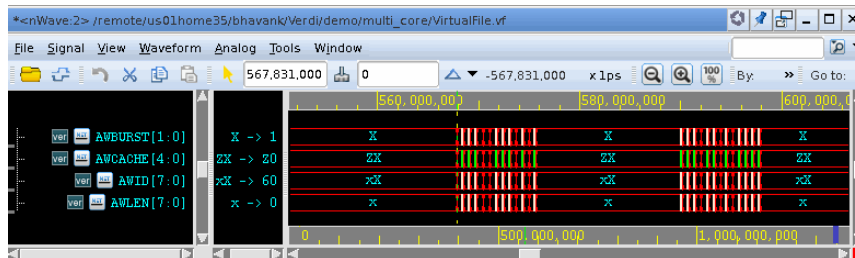
Figure 48 Workspace Launcher



6. Click the **Yes** button in the *Confirm Perspective Switch* form to confirm the perspective switch and click the **Yes** button in the *Customize Preference* form to use the recommended value for preference.

7. Get the *opcode* and *pc[31:0]* signals of the `verdiHwswDebugTop.cluster0.cpu0.instruction` scope in the *nWave* window using the **Signal -> Get Signals** command to see the executed operation code.

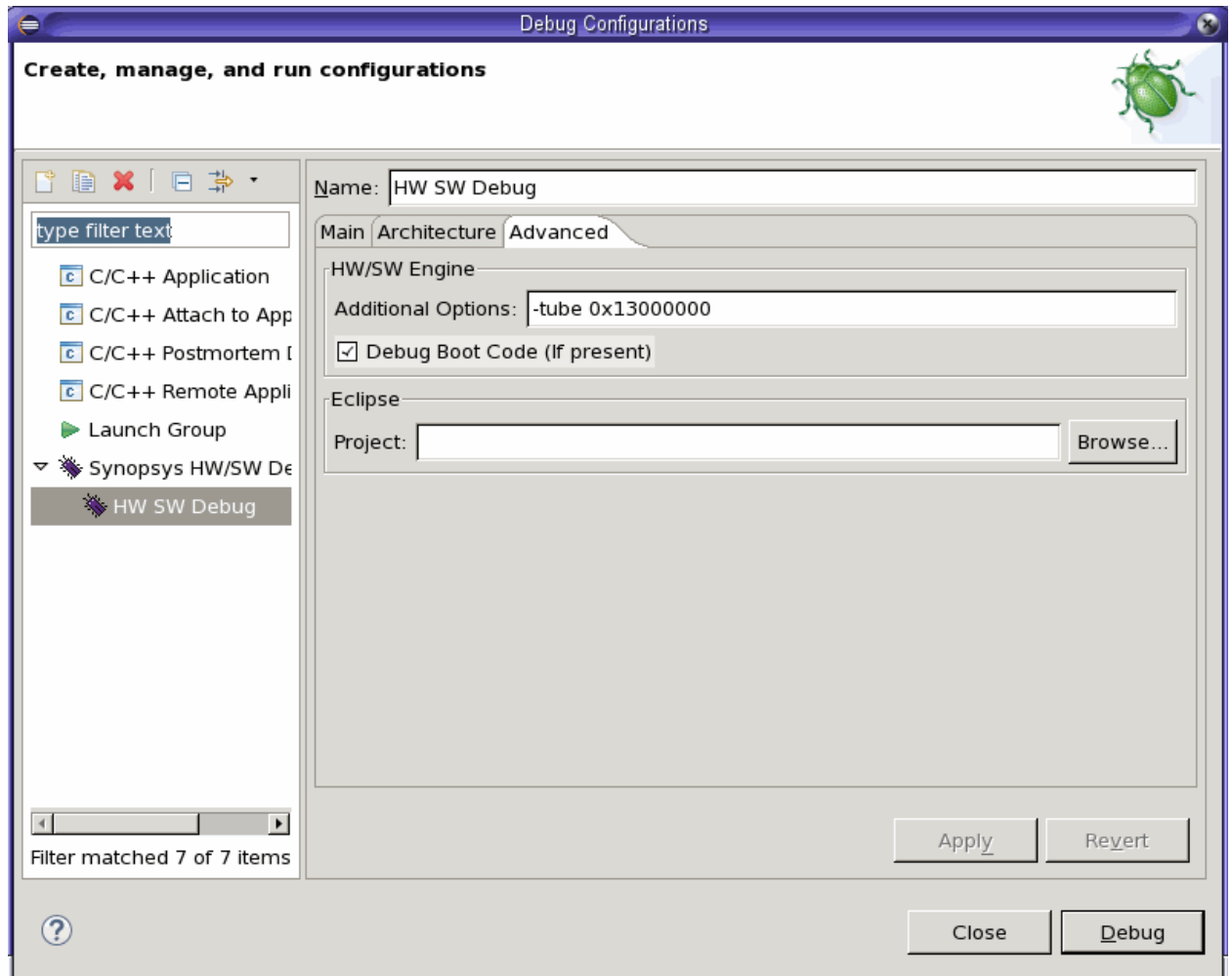
Figure 49 Get Signals in nWave



8. The `printf()` outputs are redirected to the tube (address: 0x13000000) and saved to the `tube.txt` file. The tube output can also be seen in the Eclipse output console when stepping through the `printf()`. To see the tube output, click the **Run -> Debug**

Configurations command. In the **Advanced -> Additional Options** field, enter the following value: `-tube 0x13000000`, as illustrated in .

Figure 50 Debug Configurations Form



8

Known Issues and Limitations

The following known issues exist in the HW/SW Debug feature:

- The hwsd_debug_convert tool does not support specifications of reference cycle counter signal from an FSDB to calculate timestamps, when the timestamps are based on cycle counts of an unfixed core clock.
- The hwsd_debug_convert tool does not support the interactive use model.
- It is not possible to alter the embedded core's state (registers, memory, and program flow) in *Eclipse*.